



**Tribhuvan University**  
**Faculty of Humanities and Social Science**

**Project Report on**  
**EcoTrade**  
**(Social Media Platform)**

**Submitted to**  
Department of Computer Application  
Nepal Kasthamandap College  
Kalanki, Kathmandu

***In partial fulfilment of the requirements for the bachelor's in computer  
Application***

**Submitted by**  
Nirmal Kharel  
(Reg. no. 6-2-144-15 -2020)

December, 2025

**Under the Supervision of**  
**Achyut Shah Thakuri**



**Tribhuvan University**  
**Faculty of Humanities and Social Sciences**  
**Nepal Kasthamandap College**  
Kalanki, Kathmandu  
Bachelor in Computer Applications (BCA)

**Supervisor's Recommendation**

I hereby recommend that this project prepared under my supervision by **Nirmal Kharel** entitled “**Ecotrade**” in the Partial Fulfillment of requirement for the degree of Bachelor in Computer Application is recommended for that final evaluation.

---

Achyut Shah Thakuri  
Project Supervisor  
BCA Department  
Nepal Kasthamandap College



**Tribhuvan University**  
**Faculty of Humanities and Social Sciences**  
**Nepal Kasthamandap College**  
Kalanki, Kathmandu  
**Bachelor in Computer Applications (BCA)**

**Letter of Approval**

This is to certify that this project prepared by **Nirmal Kharel** entitled “**EcoTrade**” in the Partial Fulfillment of requirement for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

|                                                                                                                               |                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <p>.....</p> <p>Achyut Shah Thakuri<br/>Lecturer<br/>BCA Department<br/>Nepal Kasthamandap College<br/>Kalanki, Kathmandu</p> | <p>.....</p> <p>Er. Sujan Poudel<br/>Head of Department<br/>BCA Department<br/>Nepal Kasthamandap College<br/>Kalanki, Kathmandu</p> |
| <p>.....</p> <p>Roshan Poudel<br/>Internal Examiner<br/>Nepal Kasthamandap College</p>                                        | <p>.....</p> <p>Roshan Tandukar<br/>External Examiner<br/>Patan Multiple College</p>                                                 |

## Abstract

This project focuses on the development of **EcoTrade**, a web-based social media platform designed to enable users to buy, exchange, or give away products within their communities. Unlike traditional marketplaces, **EcoTrade** combines social networking features with trading functionalities, allowing users to interact, follow each other, and engage in discussions about products. Key features include personalized product recommendations, advanced search and filtering, direct messaging, notifications for posts and activities, and user profile management. The platform also implements automated moderation to ensure safe and trustworthy interactions. The user interface is intuitive and responsive, offering a seamless experience for browsing, posting, and communicating. **EcoTrade** encourages community engagement through social interactions such as liking, commenting, and sharing products, while promoting sustainable practices by facilitating reuse and exchange. By integrating social networking with marketplace functionality, EcoTrade provides a unique, eco-friendly, and collaborative platform that fosters both social connections and environmentally conscious trading.

**Keywords:** *Social Media Marketplace, Community Trading, Product Exchange, Personalized Recommendations, Web Application*

## **Acknowledgement**

I would like to express my sincere gratitude to my supervisor, **Achyut Shah Thakuri** Sir, for his invaluable guidance, support, and encouragement throughout the development of this project, EcoTrade.

I am also thankful to **Nepal Kasthamandap College** for providing the necessary resources, infrastructure, and a conducive environment for completing this project. I extend my appreciation to the faculty members of the BCA Department for their valuable suggestions and academic guidance.

Finally, I would like to thank everyone who, directly or indirectly, contributed to the successful completion of this project. This experience has been highly educational, and I hope EcoTrade serves as an effective and innovative platform for sustainable, community-driven trading.

Warm regards,

**Nirmal Kharel**

# Table of Contents

|                                                                |             |
|----------------------------------------------------------------|-------------|
| <b>Supervisor's Recommendation .....</b>                       | <b>i</b>    |
| <b>Letter of Approval.....</b>                                 | <b>ii</b>   |
| <b>Abstract.....</b>                                           | <b>iii</b>  |
| <b>Acknowledgement.....</b>                                    | <b>iv</b>   |
| <b>List of Tables.....</b>                                     | <b>viii</b> |
| <b>List of Listings.....</b>                                   | <b>ix</b>   |
| <b>List of Abbreviations.....</b>                              | <b>x</b>    |
| <b>Chapter 1: Introduction .....</b>                           | <b>1</b>    |
| 1.1 Introduction.....                                          | 1           |
| 1.2 Problem Statement.....                                     | 1           |
| 1.3 Objectives .....                                           | 1           |
| 1.4 Scope.....                                                 | 2           |
| 1.5 Limitations .....                                          | 2           |
| 1.6 Methodology .....                                          | 3           |
| 1.7 Report Organization.....                                   | 4           |
| <b>Chapter 2: Background Study and Literature Review .....</b> | <b>6</b>    |
| 2.1 Background Study.....                                      | 6           |
| 2.2 Literature Review.....                                     | 7           |
| <b>Chapter 3: System Analysis and Design.....</b>              | <b>10</b>   |
| 3.1 System Analysis .....                                      | 10          |
| 3.1.1 Requirement Analysis .....                               | 10          |
| 3.1.2 Feasibility Analysis.....                                | 11          |
| 3.1.3 Object Modeling using object and class Diagram .....     | 12          |
| 3.1.4 Dynamic Modeling using Sequence Diagram .....            | 14          |
| 3.1.5 Process Modeling using Activity Diagram .....            | 16          |
| 3.2 System Design .....                                        | 18          |
| 3.2.1 Refinement of Class Diagram.....                         | 18          |
| 3.2.2 Component Diagram.....                                   | 19          |
| 3.2.3 Context Diagram.....                                     | 20          |
| 3.2.4 Deployment Diagram.....                                  | 21          |

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| 3.2.5 Wireframes .....                                                   | 21        |
| 3.2.6 Interface Design (UI Interface/Interface Structure Diagrams) ..... | 23        |
| 3.3 Algorithm Used .....                                                 | 26        |
| <b>Chapter 4 Implementaion and Testing.....</b>                          | <b>28</b> |
| 4.1 Tools Used.....                                                      | 28        |
| 4.2 Implement Details of the modules .....                               | 29        |
| 4.3 Testing.....                                                         | 33        |
| 4.3.1 Unit Testing.....                                                  | 33        |
| 4.3.2 System Testing .....                                               | 35        |
| 4.4 Result Analysis.....                                                 | 36        |
| <b>Chapter 5: Conclusion.....</b>                                        | <b>39</b> |
| 5.1 Conclusion .....                                                     | 39        |
| 5.2 Outcomes .....                                                       | 39        |
| 5.3 Future Recommendations .....                                         | 40        |
| <b>References</b>                                                        |           |
| <b>Appendices</b>                                                        |           |

## List of Figures

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| Figure 1.1: Agile Methodology .....                                                    | 3  |
| Figure 3.1: Use Case Diagram for User and Admin .....                                  | 11 |
| Figure 3.2 Class and Method diagram of EcoTrade .....                                  | 13 |
| Figure 3.3: Object Diagram of EcoTrade.....                                            | 13 |
| Figure 3.4: Sequence Diagram For Login and Feed of EcoTrade .....                      | 14 |
| Figure 3.5: Sequence Diagram For Friends, Messeging, Marketplace and Notification...   | 15 |
| Figure 3.6: Activity Diagram for Login and Post Creation in EcoTrade .....             | 16 |
| Figure 3.7: Activity Diagram For Messaging and Purchasing and Logout in EcoTrade ..... | 17 |
| Figure 3.8: Refinement of Class Diagram of Ecotrade.....                               | 18 |
| Figure 3.9: Component Diagram of EcoTrade.....                                         | 19 |
| Figure 3.10: Context Diagram of EcoTrade.....                                          | 20 |
| Figure 3.11: Deployment Diagram of EcoTrade.....                                       | 21 |
| Figure 3.12 : Wireframing of Login Page.....                                           | 21 |
| Figure 3.13: Wire Framing of Landing Page .....                                        | 22 |
| Figure 3.14: Wireframing of Product Page .....                                         | 22 |
| Figure 3.15: Wireframing of Product Card.....                                          | 23 |
| Figure 3.16: Wireframing of Product Details Card.....                                  | 23 |
| Figure 3.17: Interface Design of Login Page.....                                       | 24 |
| Figure 3.18: Interface Design of Landing Page .....                                    | 24 |
| Figure 3.19: Interface Design of Marketplace Page .....                                | 24 |
| Figure 3.20: Interface design of Product Card.....                                     | 25 |
| Figure 3.21: Interface Design of the Product Details Card.....                         | 25 |

## **List of Tables**

|                                                            |    |
|------------------------------------------------------------|----|
| Table 3.1: Gantt Chart.....                                | 12 |
| Table 4.1: Test Case for Login. ....                       | 34 |
| Table 4.2: Test Case for Post Feed.....                    | 34 |
| Table 4.3: Other Test Cases.....                           | 35 |
| Table 4.4: System Test Cases For Friend and Messaging..... | 35 |

## **List of Listings**

|                                                              |    |
|--------------------------------------------------------------|----|
| Listing 4.1: Code Snippet for Login and Registration.....    | 29 |
| Listing 4.2: Code Snippet For Ads Management.....            | 30 |
| Listing 4.3: Code Snippet For Product Ranking.....           | 31 |
| Listing 4.4: Code Snippet for Starting Conversation.....     | 31 |
| Listing 4.5 : Code Snippet to Show Conversation.....         | 32 |
| Listing 4.6: Code Snippet For Adding Friends .....           | 32 |
| Listing 4.7: Code Snippet For Accepting Friend Request ..... | 33 |
| Listing 4.8: Code Snippet showing Mutual Friendship .....    | 33 |

## List of Abbreviations

|      |                                          |
|------|------------------------------------------|
| API  | : Application Programming Interface      |
| CM   | : Content Moderation                     |
| CRUD | : Create, Read, Update, Delete           |
| CSR  | : Client-Side Rendering                  |
| CSS  | : Cascading Style Sheets                 |
| DB   | : Database                               |
| HCI  | : Human–Computer Interaction             |
| HTML | : HyperText Markup Language              |
| HTTP | : HyperText Transfer Protocol            |
| JS   | : JavaScript                             |
| JSON | : JavaScript Object Notation             |
| JWT  | : JSON Web Token                         |
| LBS  | : Location-Based Services                |
| MERN | : MongoDB, Express.js, React.js, Node.js |
| ODM  | : Object Data Modeling                   |
| REST | : Representational State Transfer        |
| SQL  | : Structured Query Language              |
| SCM  | : Source Code Management                 |
| SNS  | : Social Network Sites                   |
| SSR  | : Server-Side Rendering                  |
| UI   | : User Interface                         |
| UML  | : Unified Modeling Language              |
| URL  | : Uniform Resource Locator               |
| UX   | : User Experience                        |

# Chapter 1: Introduction

## 1.1 Introduction

EcoTrade is a web-based social media platform designed to enable users to buy, exchange, or give away products within their selected geographical area. Focused on promoting sustainability and community engagement, the platform encourages the reuse of products and goods, reducing waste and offering value for money through cost-effective access.

The system features a single-login interface that allows users to seamlessly manage all trading activities. With location-based filtering, users can view relevant listings within their area, simplifying local transactions and fostering community connections. Built using the MERN stack (MongoDB, Express.js, React.js, Node.js), EcoTrade is scalable, responsive, and ready for future expansions such as payment integration, recommendation algorithms, and mobile support.

In essence, EcoTrade is more than just a marketplace it is a community-driven solution that promotes affordability, accessibility, and environmental responsibility in goods purchasing and sharing.

## 1.2 Problem Statement

Access to essential and leisure products remains a challenge for many individuals due to high prices, limited availability, and the lack of organized platforms for sharing, exchanging, or obtaining second-hand or free items. Meanwhile, many usable products are left unused in households and institutions, resulting in unnecessary waste and missed opportunities for reuse. The Ecotrade project addresses these issues by providing a dedicated platform that enables the exchange, donation, and resale of a wide range of products, making them more accessible and promoting a culture of reuse and sustainability.

### Identified Problems:

- Existing platforms are primarily profit-driven and do not focus on product sharing or community engagement.
- Current solutions rarely support direct exchange or giveaways, which could make access easier and more inclusive.

## 1.3 Objectives

The increasing demand for affordable educational, household, and leisure resources alongside growing environmental concerns has underscored the need for platforms that

support sustainable sharing and reuse practices. Many products, such as books, electronics, and other goods, are often underutilized: purchased or acquired, then left unused despite retaining significant value. Traditional online marketplaces primarily focus on sales and often lack features for community-oriented exchange or giveaways. Ecotrade addresses this gap by providing a user-friendly, location-aware platform where individuals can buy, exchange, or give away products easily. The following objectives have been formulated to guide the development of this platform, ensuring it aligns with both user needs and sustainable practices:

- To develop a web-based that allows creating posts, live messaging, promoting business through ads.
- To develop a web-based platform that enables users to buy, exchange, or give away a wide range of products promoting affordability and accessibility.

## **1.4 Scope**

EcoTrade is designed to create a focused, community-driven platform that encourages the reuse and sharing of a wide range of products. By supporting location-based interactions and fostering a sense of community, the platform aims to provide a manageable, user-friendly environment that promotes sustainability and value for money.

- The platform facilitates buying, exchanging, and giving away products within user-selected geographic areas, making local transactions easy and efficient.
- A single login system enables users to seamlessly manage all their activities, including posting, browsing, purchasing and messaging.

## **1.5 Limitations**

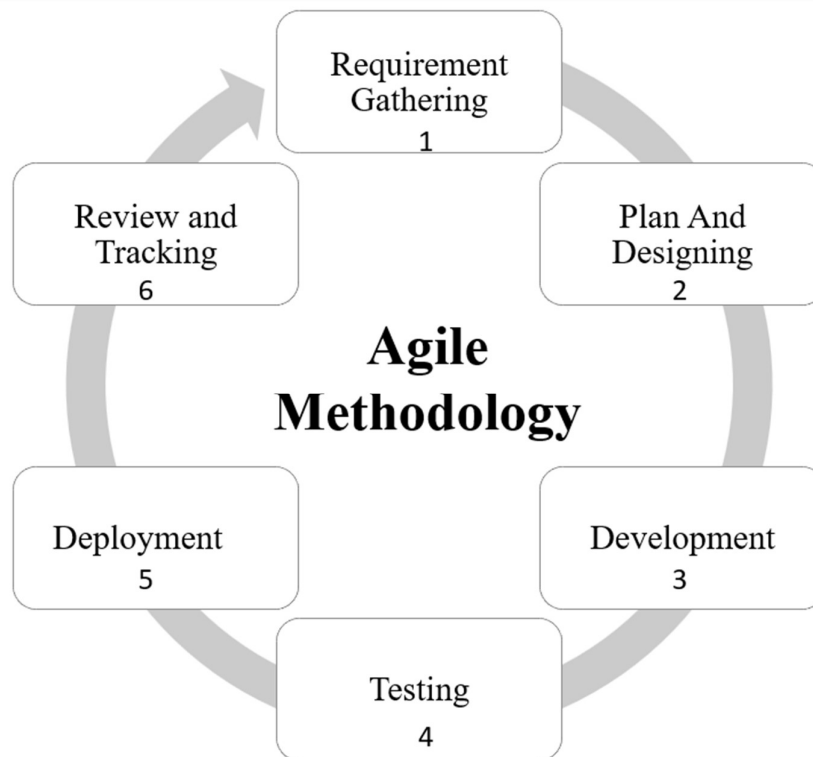
While EcoTrade is designed to provide a streamlined and community-focused platform for product sharing and exchange, there are inherent limitations that impact its initial functionality and user experience. The current scope of the project excludes certain features such as integrated payment systems and a dedicated mobile application, which may affect convenience for some users. Additionally, reliance on automated moderation and active user participation presents challenges in consistently maintaining content quality and marketplace activity. Recognizing these limitations is essential for setting realistic expectations and guiding future improvements to the platform.

- The platform's effectiveness is limited in areas with low user density, resulting in fewer available products and exchanges.

- Automated moderation cannot fully prevent inaccurate or misleading product listings, which may affect user trust.
- The platform does not handle delivery or logistics; users must arrange exchanges themselves, which can be inconvenient.
- There are limited mechanisms for resolving disputes between users regarding product condition, delivery, or transaction terms.

## 1.6 Methodology

The Agile methodology is an iterative and flexible approach to software development that emphasizes collaboration, continuous feedback, and incremental delivery. Unlike traditional development models, Agile divides work into small iterations or sprints, each producing a functional component of the system. It promotes adaptability to changing requirements, close communication between developers and stakeholders, and early testing of features. This approach ensures that the development process remains dynamic, user-centered, and capable of quickly responding to feedback or market changes.



**Figure 1.1: Agile Methodology**

In the development of EcoTrade, Agile played a crucial role by enabling rapid development and continuous improvement of multiple modules such as the marketplace, messaging, friend management, and recommendation engine. Iterative sprints allowed the team to

release functional features early, gather user feedback, and make necessary adjustments, such as improving search functionality and refining personalized recommendations. The collaborative and incremental nature of Agile also reduced the risk of major failures and made the system flexible enough to incorporate future enhancements, ultimately creating a reliable, user-friendly, and community-driven platform.

## **1.7 Report Organization**

This report is structured into five main chapters to present the study and development of the EcoTrade system in a systematic and comprehensive manner. Each chapter is designed to cover specific aspects of the project, ensuring clarity and logical flow from the conceptual stage to implementation and conclusions.

### **Chapter 1: Introduction**

This chapter provides a detailed overview of the EcoTrade project. It begins with the problem statement, highlighting the challenges in sustainable product sharing and community-based trade. The objectives section outlines the primary goals, such as promoting eco-friendly trading, enabling community interaction, and enhancing accessibility. The scope specifies the boundaries of the project, including the features covered and limitations regarding technology.

### **Chapter 2: Background Study and Literature Review**

This chapter examines the background of the sharing economy, sustainable practices, and digital trading platforms. It reviews existing literature, research studies, and similar systems to identify best practices and gaps that EcoTrade aims to address. The chapter also discusses relevant theories, concepts, and technologies used in building location-based, community-driven platforms. By analyzing previous works, this chapter establishes a strong foundation for designing a system that is both innovative and aligned with user needs.

### **Chapter 3: System Analysis and Design**

In this chapter, the detailed analysis of EcoTrade is presented. It includes functional requirements and non-functional requirements. The feasibility study evaluates technical, operational, and economic aspects of the project. System modeling is explained using UML diagrams such as use-case, class, and sequence diagrams. The chapter also covers the algorithms employed, including collaborative filtering for personalized recommendations, and describes the overall system architecture, demonstrating how different components interact to provide a seamless user experience.

#### **Chapter 4: Implementation and Testing**

This chapter focuses on the practical realization of the EcoTrade system. It introduces the tools, technologies, and frameworks used. A detailed explanation of each module is provided, covering functionalities like product listing, user authentication, messaging, and recommendation engine. The chapter also discusses the implementation process, integration of components, and challenges faced. Furthermore, it explains the testing procedures, including unit testing, integration testing, and system testing, ensuring that all features work as intended and meet the project requirements.

#### **Chapter 5: Conclusion**

The final chapter summarizes the entire study and development process of EcoTrade. It highlights the key outcomes, such as improved product accessibility, enhanced community interaction, and promotion of sustainable practices. The chapter also reflects on the system's effectiveness in meeting the objectives and identifies limitations encountered during development. Finally, it suggests potential future enhancements, such as integrating mobile applications, adding advanced AI-based recommendations, and expanding community features to further improve usability and impact.

## **Chapter 2: Background Study and Literature Review**

### **2.1 Background Study**

As digital transformation accelerates, online platforms that enable the reuse and sharing of physical goods have gained importance in fostering sustainable consumption and community engagement. EcoTrade aims to address these issues by facilitating the localized exchange, sale, and giveaway of products through a user-friendly web application. In recent years, the concept of the sharing economy has expanded beyond traditional sectors such as transportation and accommodation to include goods like books [1]. This movement encourages users to share, lend, or exchange items to maximize resource utilization and reduce environmental impact. However, many existing platforms focus mainly on sales or donations without integrating comprehensive exchange features or geographical relevance, which can limit community involvement and the overall efficiency of products reuse. [2] EcoTrade's focus on location-based interactions aligns with research indicating that proximity plays a critical role in peer-to-peer exchanges. Localized marketplaces reduce transportation costs and carbon footprints while encouraging stronger community ties. The platform simplifies the user experience and leverages niche expertise, allowing for better matching of supply and demand and more effective moderation.

Sustainability considerations are central to EcoTrade's design. Beyond simply facilitating transactions, the platform incorporates environmental impact awareness by estimating the positive effects of reuse, such as carbon emission reductions. This feature serves to motivate users by making the benefits of their actions tangible, supporting wider goals of environmental responsibility and resource conservation. [3]

Technologically, EcoTrade leverages the MERN stack (MongoDB, Express.js, React.js, Node.js), a modern and flexible development environment suitable for building scalable and maintainable web applications. The choice of this stack ensures that the platform can efficiently handle real-time updates, user interactions, and data management, essential for providing a smooth and responsive user experience. [4]

## 2.2 Literature Review

Online marketplaces and peer-to-peer platforms have been widely studied as mechanisms for resource sharing and community engagement. Platforms like eBay and OLX pioneered the buying and selling of second-hand goods, but their broad focus often results in less tailored experiences for specific user needs such as book trading. Research suggests that specialized platforms can offer superior user satisfaction by addressing domain-specific challenges and fostering stronger communities [5].

Studies on sharing economy applications highlight the importance of trust and safety in encouraging participation. Automated moderation systems and user rating mechanisms have been identified as key factors that help maintain platform integrity . EcoTrade's approach to automated content moderation aligns with these findings, aiming to minimize manual intervention while preserving quality and user trust. Location-based services (LBS) have been increasingly integrated into sharing platforms to improve relevance and user convenience. Literature shows that proximity not only reduces logistical challenges but also enhances social connections among users, fostering community cohesion. [6]

Environmental sustainability is a growing area of interest in the literature on digital marketplaces. Several studies demonstrate that platforms enabling the reuse of goods contribute significantly to waste reduction and resource efficiency . However, few platforms actively communicate this impact to users.

EcoTrade's environmental impact calculator is inspired by research advocating for real-time feedback as a motivator for sustainable behavior change. Finally, technology stack choices play a critical role in platform performance and scalability. The MERN stack is recognized for its full-stack JavaScript environment, allowing for efficient development and real-time data handling and case studies on MERN-based applications show advantages in flexibility and community support, making it a fitting [7].

The article "Impact of Social Media: Advantages and Disadvantages in 2026" provides a comprehensive analysis of the dual nature of social media's effect on modern life . On one hand, social platforms are powerful tools that foster improved communication and global connectivity, serve as a vast educational resource, and create valuable marketing and employment opportunities by facilitating brand growth and professional networking . Furthermore, social media plays a crucial role in civic engagement, supporting social movements and advocacy through rapid information dissemination and community building . Conversely, the platforms pose significant challenges: excessive use is strongly

linked to mental health issues such as anxiety, depression, and low self-esteem due to the prevalent comparison culture . Users are also exposed to severe risks related to privacy concerns, the rampant spread of misinformation, and the potential for cyberbullying and harassment . Ultimately, the article suggests that while social media offers substantial benefits, its use must be balanced and controlled to avoid detrimental effects like addiction and distraction, which negatively impact productivity [8].

The central argument of Mangold and Faulds is that social media fundamentally changed the rules of engagement, requiring marketers to view it as a "hybrid element" a mixture of traditional promotional tools (like advertising and public relations) and a unique, new medium. Traditional marketing relied on a one-to-many communication model, where the company pushes a message to a mass audience. Social media, leveraging Web 2.0, introduced an interactive, peer-to-peer (many-to-many) model . This new structure effectively grants control over content to consumers, who participate in the development and dissemination of marketing messages, primarily through electronic Word-of-Mouth (eWOM). The strategic implication is that firms must shift their focus from merely *controlling* the message to actively encouraging and facilitating customer participation to amplify positive brand messages and manage negative sentiment . [9]

Kietzmann introduce the influential "honeycomb framework," which outlines seven key functional building blocks of social media: identity, conversations, sharing, presence, relationships, reputation, and groups. Each block represents a core social function that platforms enable, and the model emphasizes that different platforms prioritize different combinations of these functions. By focusing on underlying social mechanisms rather than platform labels, the framework provides a structured way to analyze how features shape user engagement and behavior. This perspective helps managers, designers, and researchers understand why certain platforms succeed for instance, Instagram thrives on sharing and identity, while WhatsApp focuses on conversations and presence. The framework therefore acts as both a conceptual map of the social media landscape and a diagnostic tool for evaluating platform capabilities across diverse contexts.

Beyond describing functional components, Kietzmann and colleagues highlight the broader ethical, managerial, and governance implications of social media design. They stress that features emphasizing identity or sharing must address privacy and data protection, while platforms built on conversations or reputation require effective moderation to prevent misuse and misinformation. By discussing these risks alongside opportunities, the paper encourages the creation of more responsible and human-centered social ecosystems. It also

acts as a bridge between fields such as marketing, HCI, and communication studies by offering insights that guide both scholarly inquiry and practical strategy development. The framework's clarity, adaptability, and relevance have made it widely used for analyzing existing platforms and informing the design of new digital social experiences. [10]

Kaplan and Haenlein offer a compact practitioner-oriented taxonomy and define social media from a marketing/business perspective. They classify social media types (blogs, collaborative projects, content communities, social networking sites, virtual game worlds, virtual social worlds) and discuss strategic implications for organizations and managers. The article's contribution is an operational framework mapping types of social media to marketing and communication objectives, along with cautions about metrics, control, and managing user-generated content. While less focused on sociological theory than Boyd & Ellison, it is highly influential for bridging academic concepts with managerial practice and remains heavily cited in applied research on social media strategy. [11]

Boyd and Ellison produce a precise conceptualization of Social Network Sites (SNSs), defining them around three core functions: profile creation within a bounded system, articulation of a list of connections, and the ability to view and traverse connections. The work synthesizes early SNS history, clarifies terminological confusion (social network sites vs. networks), and maps key theoretical questions for future research identity management, privacy, networked publics, and social capital. Its strength lies in giving researchers a clear set of affordances and empirical questions rather than a narrow technological definition; by focusing on social practice rather than technology only, the paper became the canonical citation for subsequent empirical and theoretical studies. It also highlights methodological concerns (qualitative vs. large-scale data) that foreshadow debates about platform data access and ethics. [12]

## **Chapter 3: System Analysis and Design**

### **3.1 System Analysis**

EcoTrade is developed as a responsive web application using the MERN, following the Agile development approach. The system supports a user-centric design with features including product listings, location-based filtering, and environmental impact tracking. System analysis focuses on thoroughly understanding the current problems and determining how EcoTrade can address them effectively.

#### **3.1.1 Requirement Analysis**

Requirement analysis involves collecting detailed information on what the system should do from users, developers, and other stakeholders

##### **Functional Requirements**

Functional Requirements define the core functionalities the system must support. Some key functional requirements of the project are as below:

- User Registration/Login (Single Sign-On)
- Create and Post product Listings
- Custom Product Filter
- Post Creation and Interaction
- Product Trading Options (Buy/Exchange/Giveaway)
- Advertisement Management
- In-app Messaging



**Figure 3.1: Use Case Diagram for User and Admin**

### Non Functional Requirements

Non-Functional Requirements define how the system should perform:

- application should offer a responsive interface that adjusts layout across screen sizes for improved web usability.
- provide low-latency responses for smooth user experience during interactions.
- Automated keyword-based moderation should flag inappropriate content without human input during posting.
- must provide low-latency responses for smooth user experience during interactions.

### 3.1.2 Feasibility Analysis

Feasibility analysis evaluates whether the project is viable in terms of technology, operation, and economy.

#### Operational Feasibility

The system aims to deliver user-friendly interfaces, intuitive flows, and automated features, making operations seamless. Admin tools further ensure smooth backend management.

### Technical Feasibility

EcoTrade leverages the MERN stack (MongoDB, Express.js, React, Node.js), which provides a modern, scalable, and efficient development framework. Cloud services and containerization also enhance deployment and maintainability.

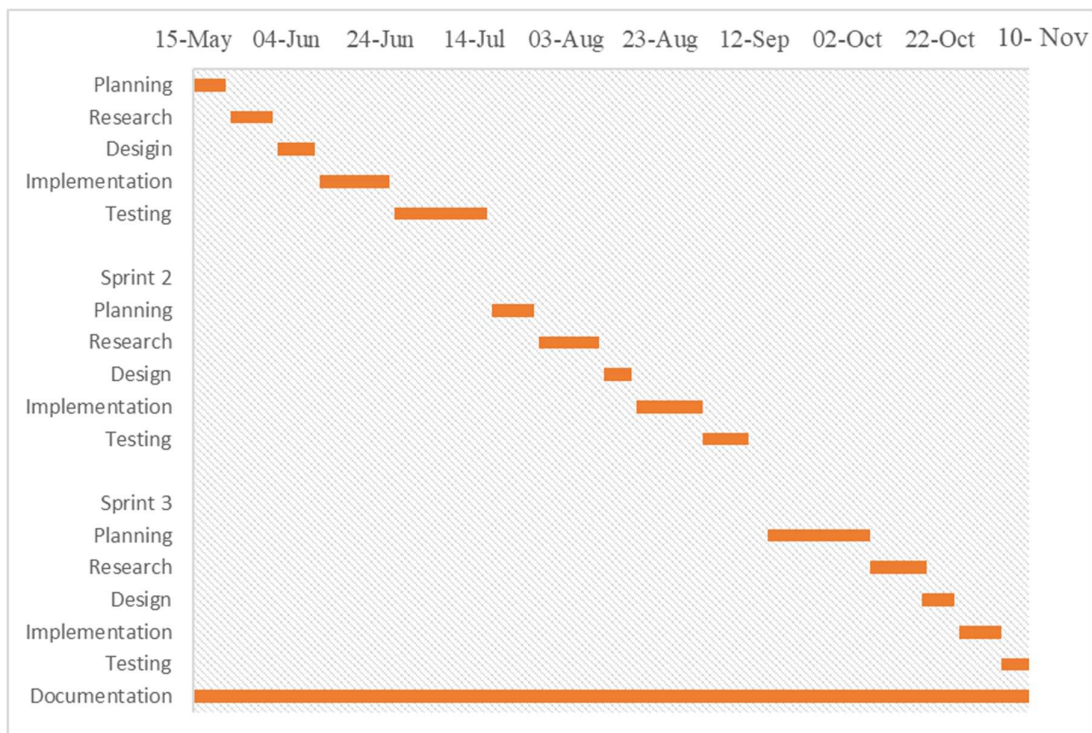
### Economic Feasibility

Open-source technologies reduce initial costs. The sustainable model of product reuse ensures long-term benefits with minimal recurring expenses.

### Schedule Feasibility

The Gantt Chart outlines the project timeline, ensuring transparency and tracking progress across five months. It helps the development team monitor task durations and dependencies effectively.

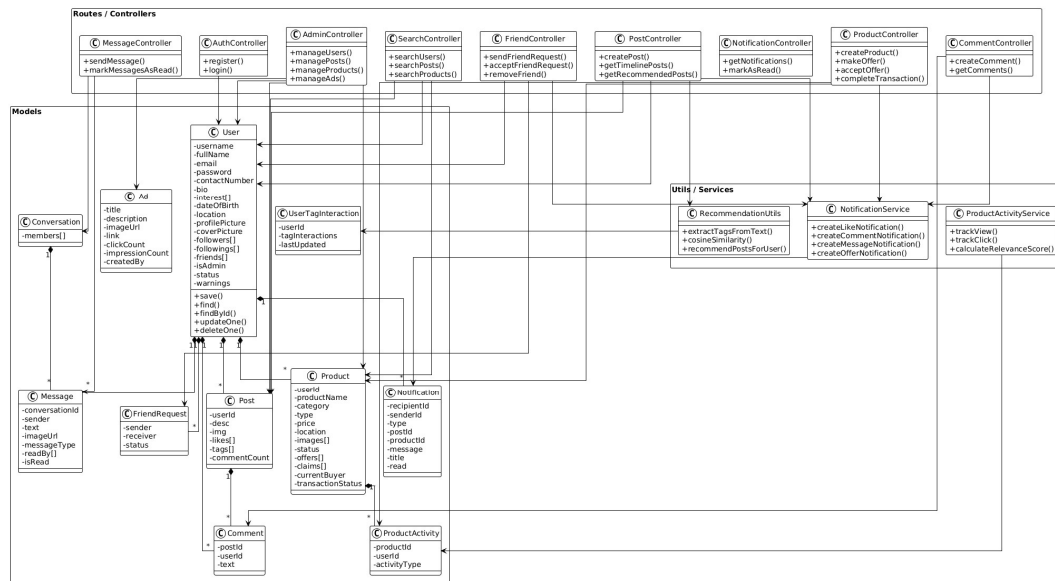
**Table 3.1: Gantt Chart**



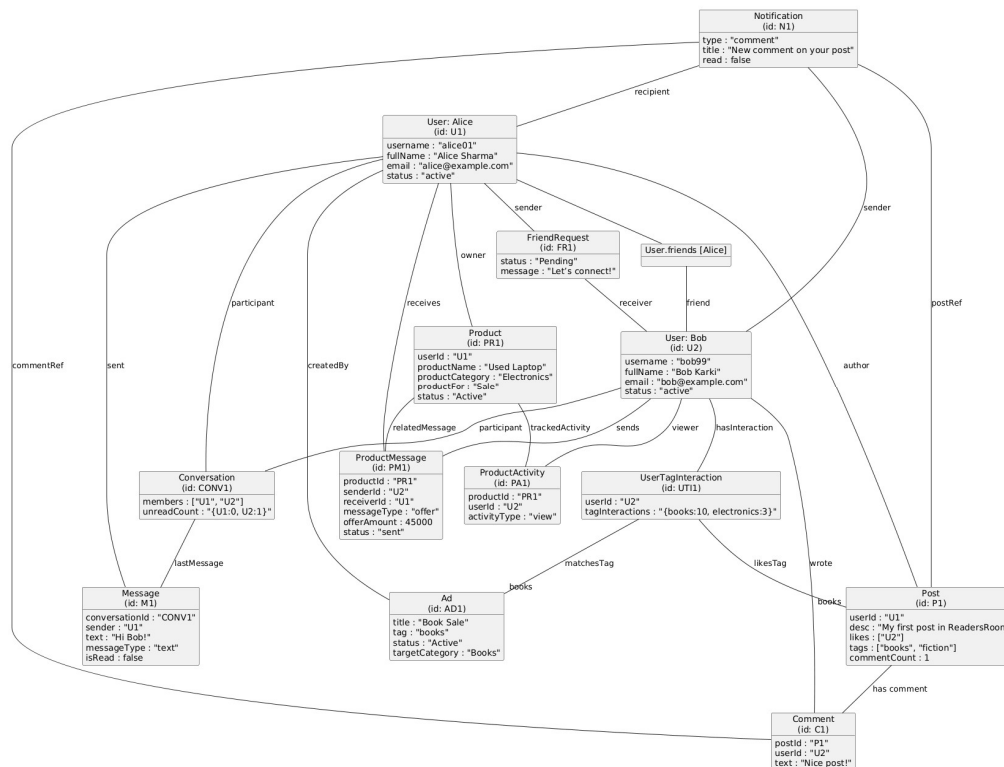
### 3.1.3 Object Modeling using object and class Diagram

The class diagram represents the core structure of the EcoTrade social media marketplace, showing the main classes and how they interact. At the center is the User class, which connects to posts, products, comments, messages, and friend requests. Users can create posts, list products for sale or exchange, comment, like, and communicate through

conversations or product-specific messages. Each product contains details such as price, condition, and category, while posts contain images, descriptions, and comments.



**Figure 3.2 Class and Method diagram of EcoTrade**

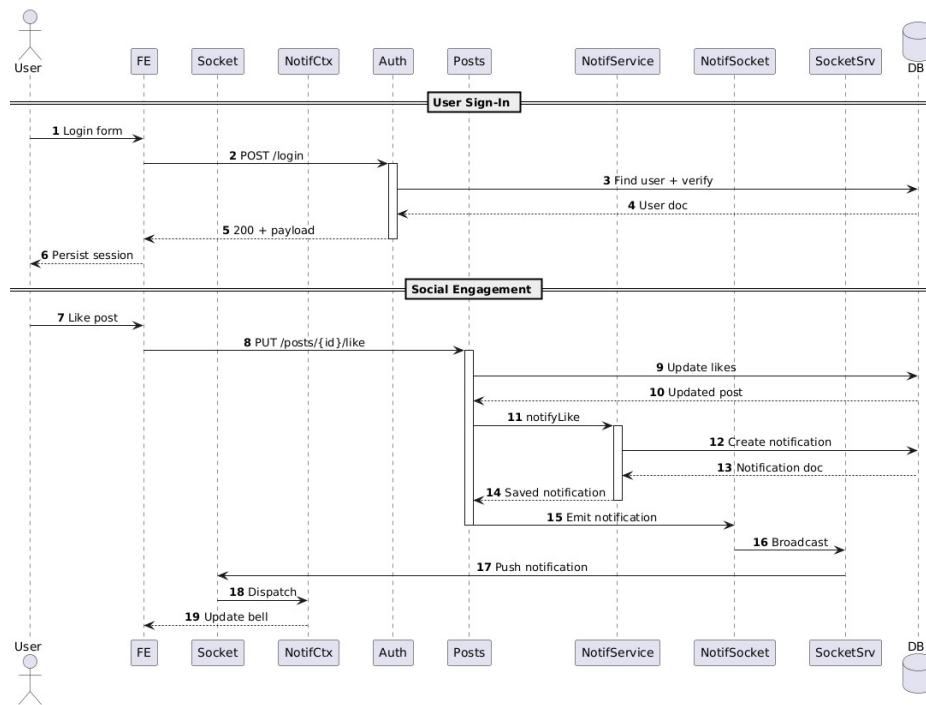


**Figure 3.3: Object Diagram of EcoTrade**

This figure represents the Class and Method Diagram of the EcoTrade system, a social media-based marketplace for buying, exchanging, and giving away books. The diagram shows how EcoTrade is structured into classes (models) with their attributes and methods,

and how they are accessed through controller classes. Core classes such as User, Product, Post, Comment, Message, FriendRequest, Notification, and Conversation define the main data entities of the system, with methods like `save()`, `find()`, `updateOne()`, and `deleteOne()` representing database operations. The User class is central, as users can create posts and products, send messages, make offers, send friend requests, and receive notifications. Controller classes such as AuthController, ProductController, PostController, MessageController, and FriendController expose methods like `createProduct()`, `makeOffer()`, `sendMessage()`, and `sendFriendRequest()` that implement system functionalities by invoking model methods. Supporting service classes like RecommendationUtils, NotificationService, and ProductActivityService contain methods for recommendation logic, notification handling, and activity tracking.

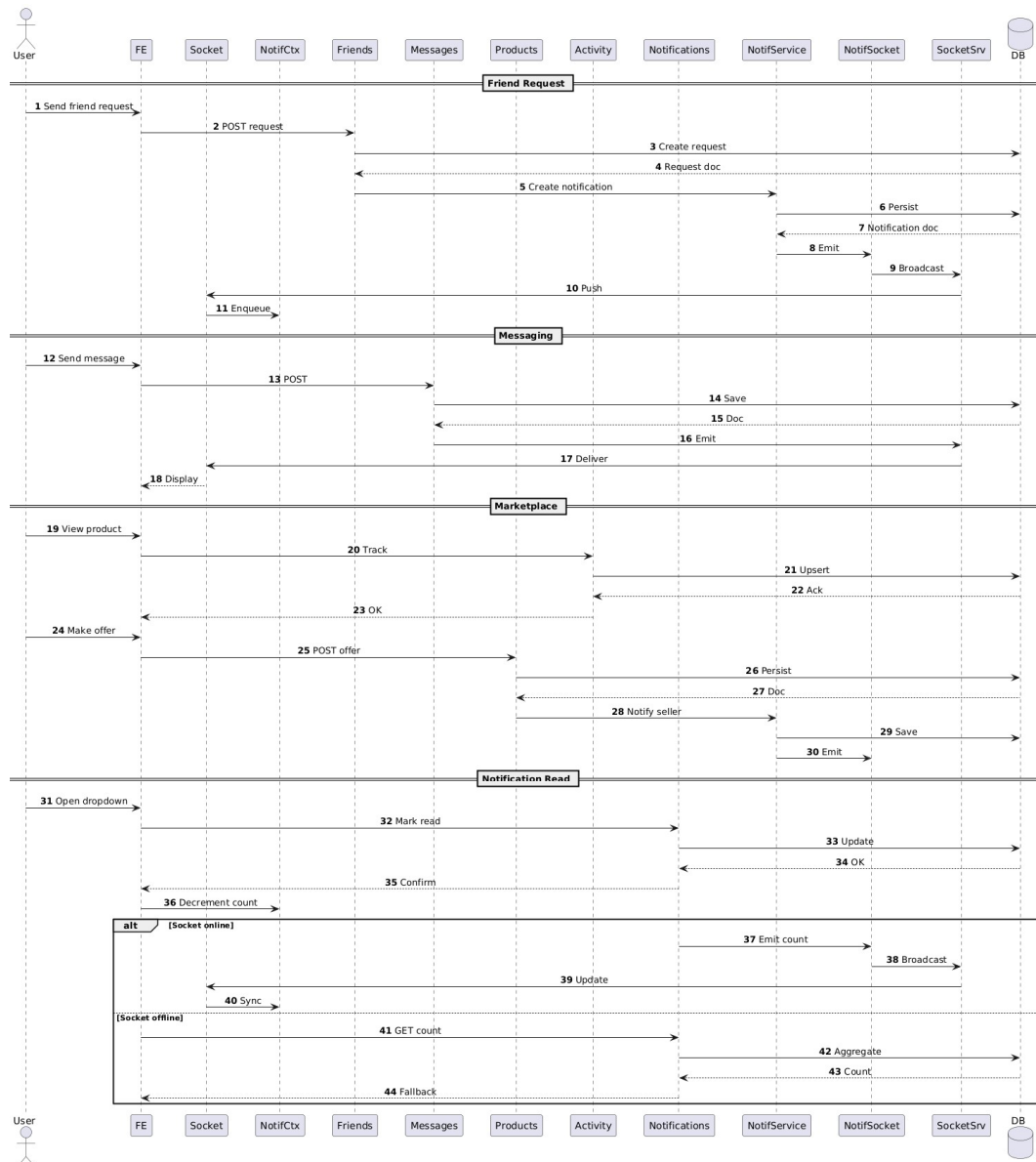
### 3.1.4 Dynamic Modeling using Sequence Diagram



**Figure 3.4: Sequence Diagram For Login and Feed of EcoTrade**

The sequence diagram above shows how the system handles user login and social engagement through coordinated interactions between the front-end, backend services, sockets, and the database. During login, the front-end sends the user's credentials to the Auth service, which verifies them with the database and returns session data for the front-end to store. When a user likes a post, the Posts service updates the like count, creates a notification through the Notification Service, saves it, and broadcasts it via sockets. The

front-end then receives the push notification and updates the UI accordingly, demonstrating seamless authentication, post interaction, and real-time alerts.

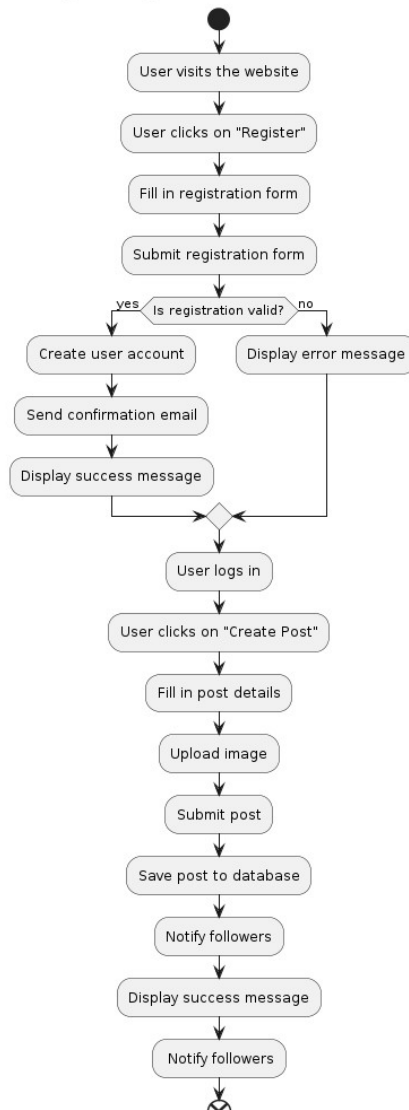


**Figure 3.5: Sequence Diagram For Friends, Messeging, Marketplace and Notification**

The sequence diagram above summarizes how the system handles four user interactions friend requests, messaging, marketplace actions, and notification reads by coordinating between the front-end, backend services, sockets, and the database. When a user sends a friend request or message, the system saves it, generates notifications, and broadcasts updates in real time. Marketplace actions like viewing a product or making an offer are tracked and also trigger notifications to sellers. Finally, when the user reads notifications, the system updates their read status and adjusts the unread count using either real-time

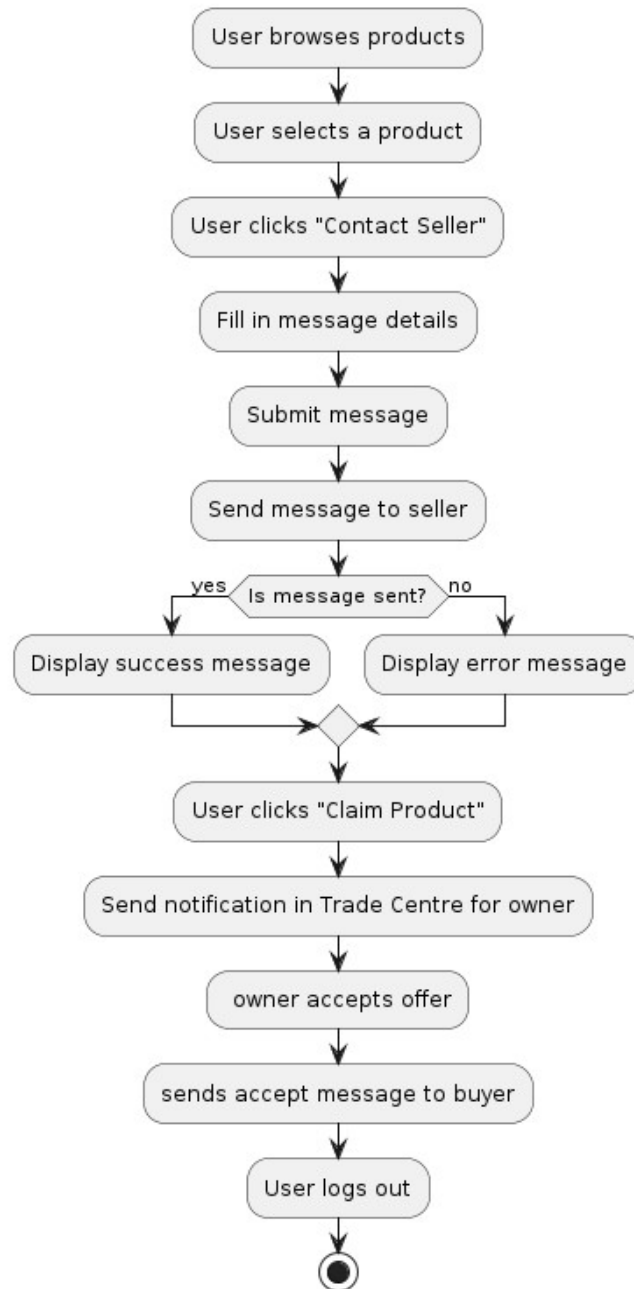
socket updates or a fallback API call. Overall, the diagram shows how different components work together to provide instant, synchronized user experience.

### 3.1.5 Process Modeling using Activity Diagram



**Figure 3.6: Activity Diagram for Login and Post Creation in EcoTrade**

This activity diagram shows the flow of a user registering on a website and creating a post. First, the user visits the site, fills out a registration form, and the system checks if the information is valid. If valid, an account is created, a confirmation email is sent, and a success message appears; otherwise, an error is shown. After logging in, the user creates a post by entering details and uploading an image. The system validates the post if valid, it is saved, followers are notified, and a success message is displayed; if not, an error message is shown.



**Figure 3.7: Activity Diagram For Messaging and Purchasing and Logout in EcoTrade**

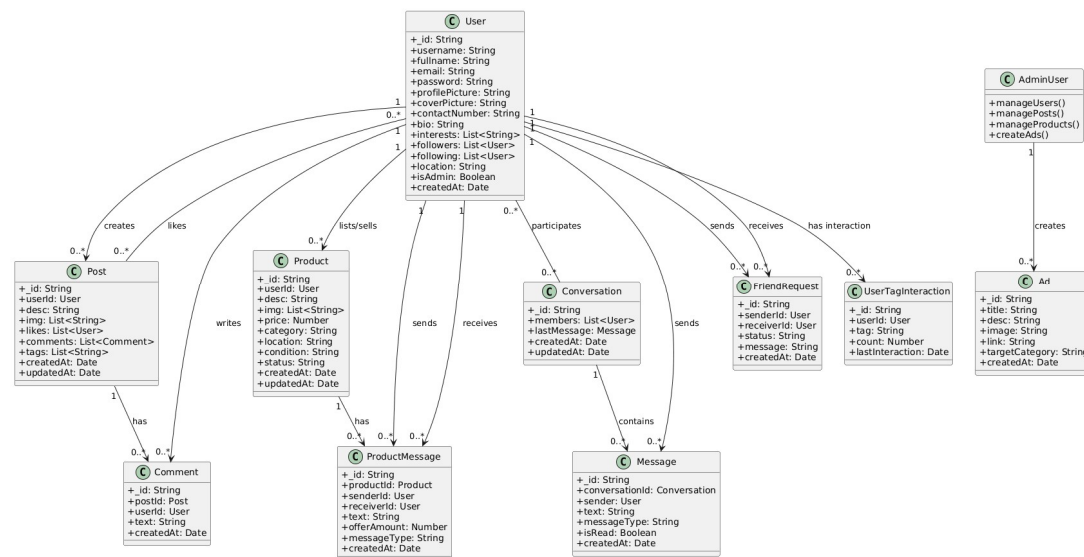
This activity diagram shows the steps a user takes to message a seller, purchase a product, and log out. The user selects a product, contacts the seller, fills in message details, and submits the message. The system checks if the message is successfully sent showing either a success or error message. Next, the user chooses to purchase the product, enters payment details, and the system validates the payment. If valid, the payment is processed and a purchase confirmation is displayed; if not, a payment error is shown. Finally, the user logs out, completing the flow.

## 3.2 System Design

To clearly represent all the system requirements in a graphical way, various diagrammatic designs of the system were prepared, which are as follows:

### 3.2.1 Refinement of Class Diagram

This class diagram represents a fully refined structure of a Ecotrade, showing all the major entities, their attributes, and how they interact with each other. At the core of the diagram is the User class, which connects to almost every feature of the system. A user can create posts, write comments, like posts, list products for sale, send and receive product messages, join conversations, send friend requests, and interact with tags. This central role reflects how users drive all social and marketplace activities. Each relationship is clearly labeled with multiplicity (such as 1 to many, many to many), showing exactly how objects relate for example, one user can create many posts, a post can have many comments, and a user can send many friend requests.

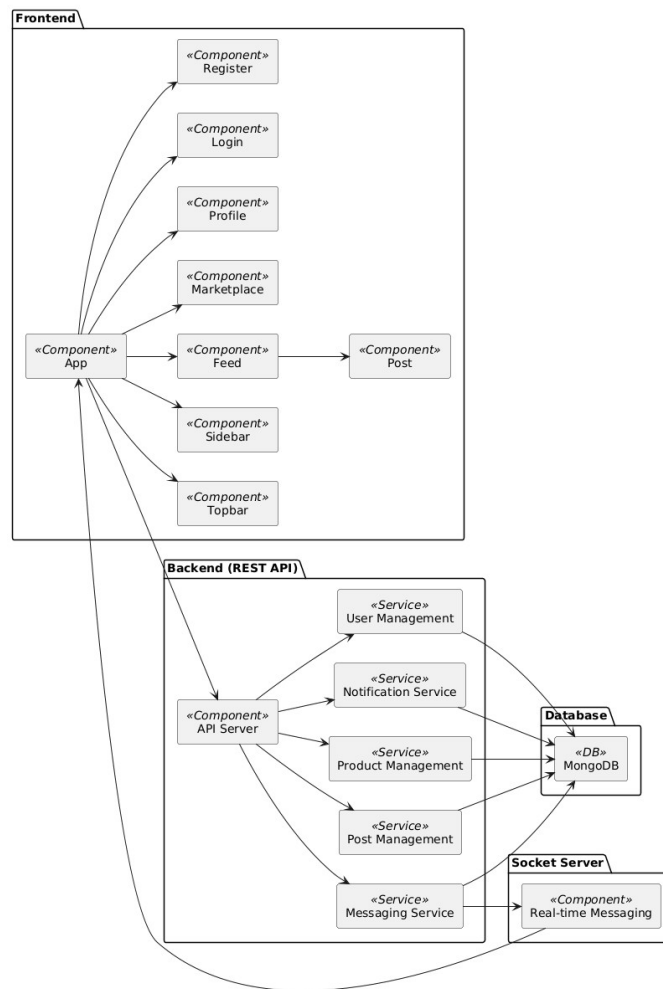


**Figure 3.8: Refinement of Class Diagram of Ecotrade**

The diagram also includes marketplace-specific features. A Product is listed by a user and can have multiple product messages between buyers and sellers. The ProductMessage class captures negotiations or inquiries about a product. Similarly, the Conversation and Message classes enable private messaging between users, with a conversation containing many messages. Friendships are handled through the FriendRequest class, where a user can send and receive multiple requests. On the analytics side, the UserTagInteraction class tracks how often a user interacts with specific tags, supporting features like personalized

recommendations. Additionally, the AdminUser class represents system administrators who manage platform content and create advertisements. The Ad class stores promotional content displayed to users. Overall, the diagram gives a complete view of all social media, messaging, product listing, marketplace communication, admin functions, and user interaction mechanisms in the system, showing how each part is connected and works together.

### 3.2.2 Component Diagram

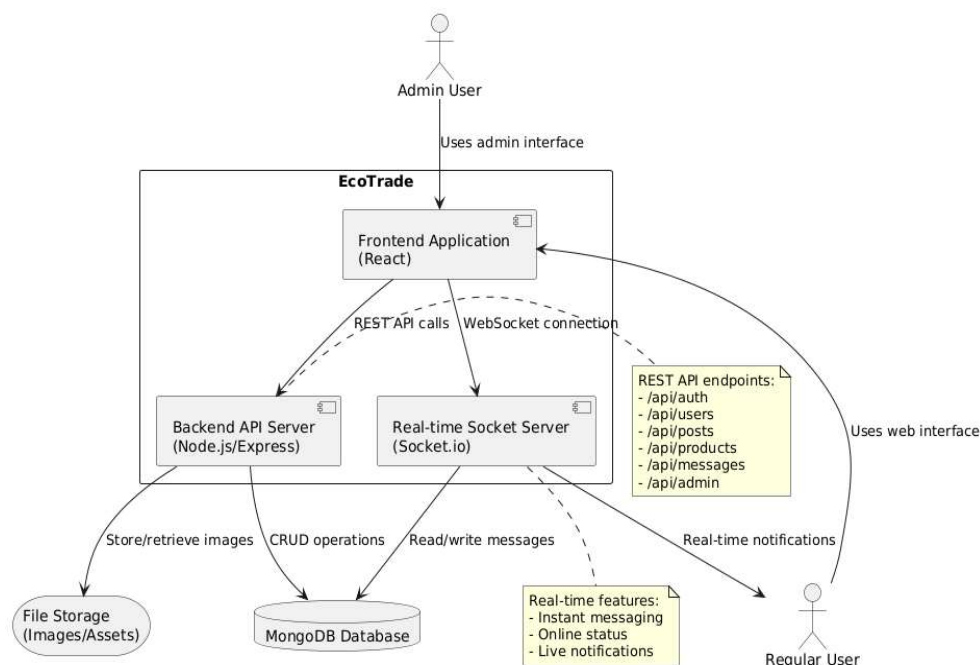


**Figure 3.9: Component Diagram of EcoTrade**

The above component diagram illustrates how the system is organized into four main layers: the Frontend, the Backend REST API, the Database, and the Socket Server. The Frontend contains several user interface components such as Register, Login, Profile, Marketplace, Feed, Post, Sidebar, and Topbar all coordinated by the main App component, which communicates directly with the backend. The Backend section is centered around

the API Server, which receives requests from the frontend and delegates them to specialized services like User Management, Notification Service, Product Management, Post Management, and Messaging Service. Some backend services, especially Messaging and Notification, also communicate with the Socket Server, which is responsible for real-time features such as live chat and instant notifications. The diagram shows how data flows between the components: the Frontend sends API calls to the API Server, backend services process the logic and communicate with MongoDB, and the Socket Server pushes real-time updates back to the user interface.

### 3.2.3 Context Diagram

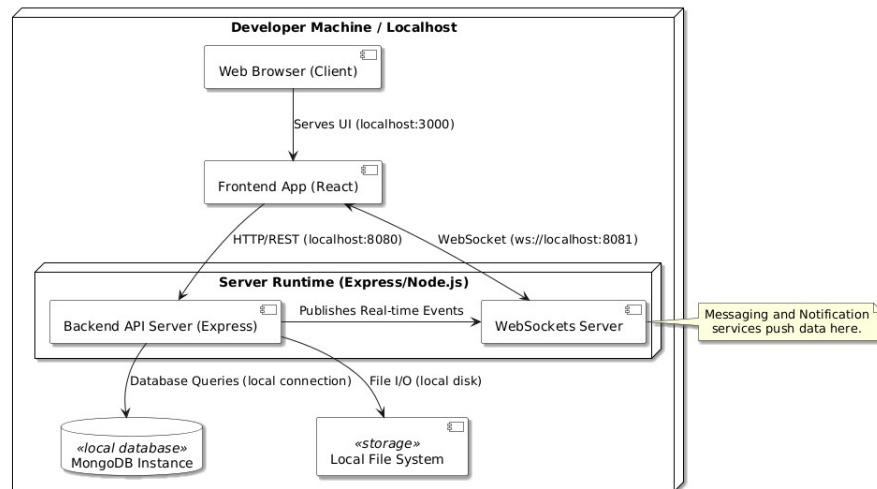


**Figure 3.10: Context Diagram of EcoTrade**

The above figure is a context diagram of EcoTrade, illustrating the system's interactions with users, admins, and external systems. Regular users access the platform through the frontend web interface, while admin users manage the system via the admin interface. The frontend application (React) communicates with the backend API server (Node.js/Express) through REST API calls and connects to the real-time Socket.io server for live messaging, notifications, and online status updates. The backend interacts with the MongoDB database for storing users, posts, products, and messages, and with file storage for images and assets. Key API endpoints include /api/auth, /api/users, /api/posts, /api/products, /api/messages, and /api/admin, enabling authentication, user management, product operations, and messaging, while the socket server handles instant communication and real-time updates.

### 3.2.4 Deployment Diagram

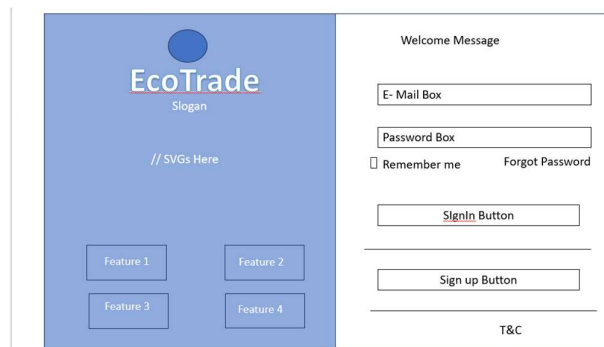
This deployment diagram shows how the EcoTrade MERN-stack application runs in a local development environment. The user interacts with the system through a web browser, which loads the React frontend served on localhost:3000.



**Figure 3.11: Deployment Diagram of EcoTrade**

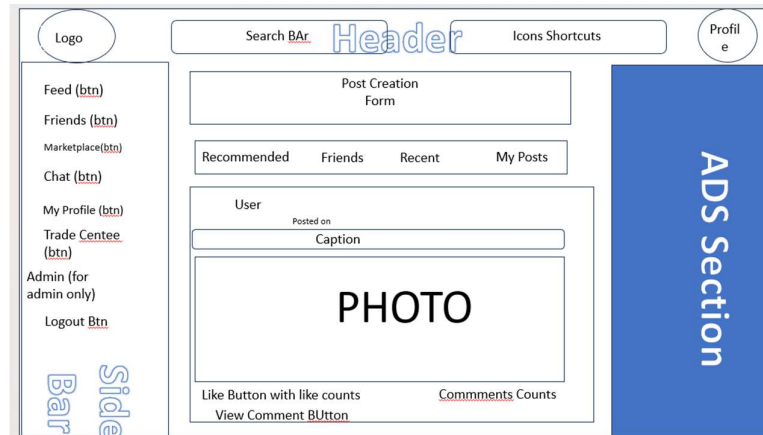
The frontend communicates with the backend Express/Node.js server using HTTP/REST (localhost:8080) for API requests and WebSockets (ws://localhost:8081) for real-time updates such as messaging and notifications. The backend server handles business logic, processes requests, and publishes real-time events to the WebSocket server. It also performs database operations using a local MongoDB instance and handles file storage through the local file system. All components run on the developer's machine under the local environment.

### 3.2.5 Wireframes



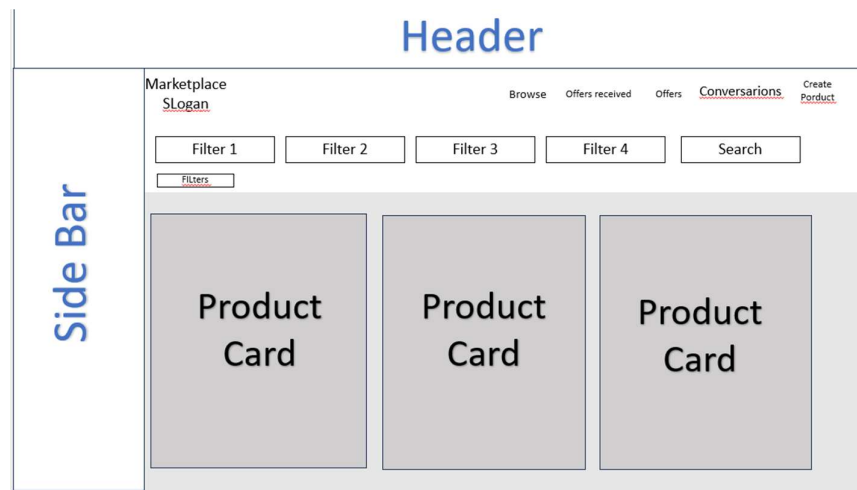
**Figure 3.12 : Wireframing of Login Page**

This wireframe shows a two-part login page: the left side displays the brand, slogan, and features, while the right side provides the login form with email, password, sign-in/sign-up options, and T&C.



**Figure 3.13: Wire Framing of Landing Page**

This wireframe shows landing page of Ecotrade. A left sidebar contains navigation buttons like Feed, Friends, Marketplace, Chat, Profile, Trade Center, Admin, and Logout. The top header includes the logo, search bar, shortcuts/icons, and user profile. The center area displays the post creation form and filtered tabs (Recommended, Friends, Recent, My Posts), along with the post with user info, caption, photo, likes, comments, and a view-comments button. The right side is reserved for an ads section.



**Figure 3.14: Wireframing of Product Page**

This wireframe shows a product listing page with a header, sidebar, navigation links, multiple filter options, a search bar, and a grid of product cards displayed in the main area.



**Figure 3.15: Wireframing of Product Card**

This wireframe shows a simple product card: it displays the seller's info, listing type and status, the product name, a large product image with an image count, and two action buttons View Details and Message.



**Figure 3.16: Wireframing of Product Details Card**

This wireframe represents a product details card: it shows the product name, asked value, listing and status tags, a large image area, and three info sections Product Details, Pricing & Exchange, and Contact & Timeline along with View Details and Message action buttons.

### 3.2.6 Interface Design (UI Interface/Interface Structure Diagrams)

Below are the Interfaces Designed based on the wireframes in the EcoTrade

## Login page

Login Page for the Registered User Login

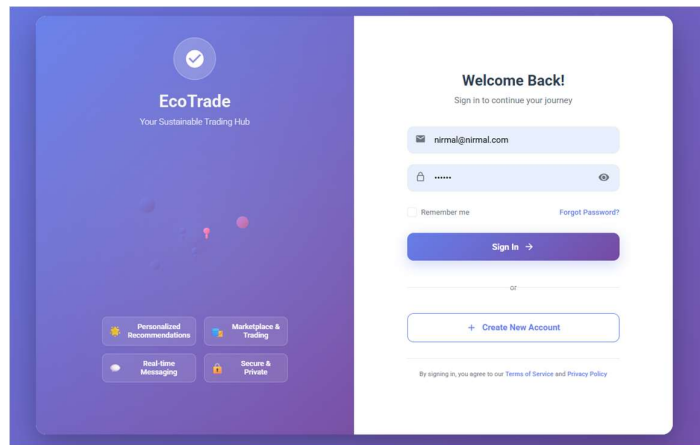


Figure 3.17: Interface Design of Login Page

## Landing Page

First Landing Page of the Ecotrader with header on top, left side bar with various modules and features central part for feeds and posts and right side for the advertisements.

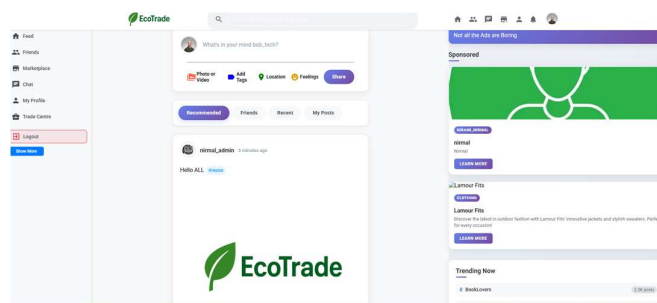


Figure 3.18: Interface Design of Landing Page

## Marketplace Page

Marketplace page showing the list of the product based on the recommendation score along with recommendation score, listing type, view details button and message seller button that redirects to the sending message.

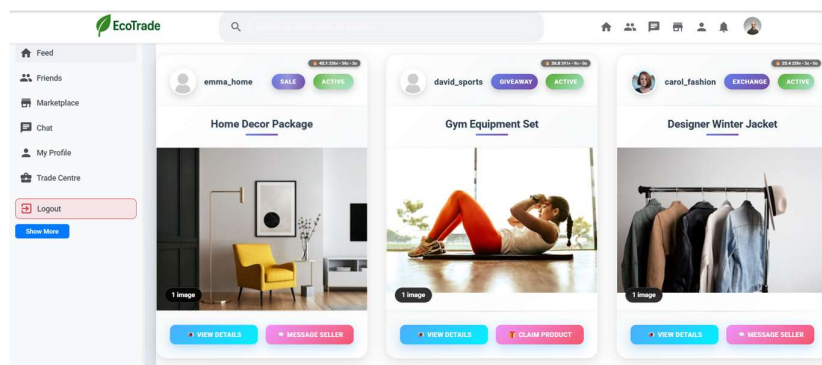


Figure 3.19: Interface Design of Marketplace Page

## Products card

Product card with the id of owner, recommendation score of the top right corner of the card, and options to view details and message the seller(owner).

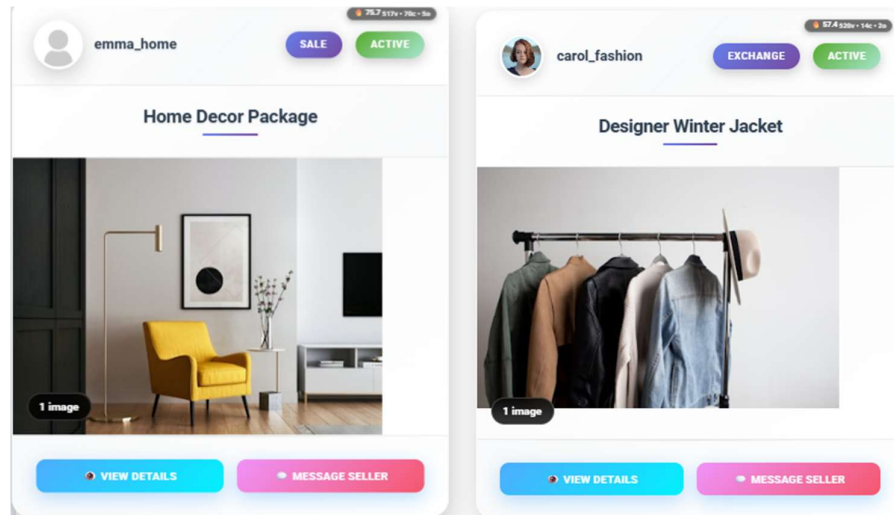


Figure 3.20: Interface design of Product Card

## Product Details Card

Detailed View of product with all the product images , and the details of the product in different segment.

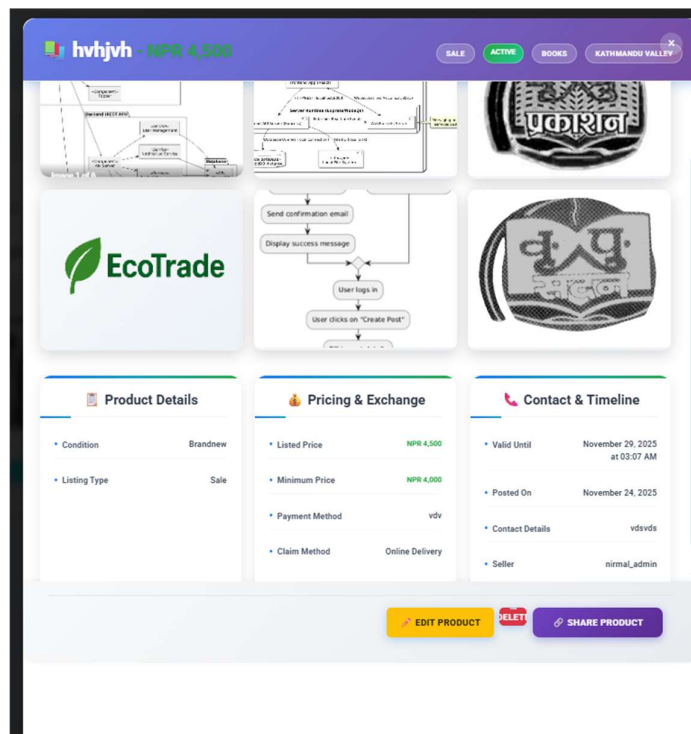


Figure 3.21: Interface Design of the Product Details Card

### 3.3 Algorithm Used

To deliver personalized recommendations, EcoTrade can implement filtering algorithm whose detailed explanation is as below:

#### i. Personalized Tag-Based Collaborative Filtering for Post Recommendations

##### Concept

Recommends posts to users based on their personal tag preferences and the behavior of similar users. The system learns which tags a user interacts with most and combines this with collaborative filtering to suggest relevant posts.

##### Mechanics

##### 1. Track User Tag Interactions

- For each user, maintain a tag frequency vector.
- When a user likes, comments, or interacts with a post, increment the count for each tag on that post.
- Example: If a user likes a post tagged #books and #reuse, increment both in their tag vector.

##### 2. Compute User Similarity

- Periodically, compute cosine similarity between users' tag vectors.
- Users with similar tag vectors are considered "similar users."

##### 3. Score Posts for Recommendation

- **Content-based:** Score posts higher if they contain tags user frequently interacts with.
- **Collaborative:** Score posts higher if similar users have interacted with them.
- Combine both scores (e.g., weighted sum).

##### 4. Recommend Top-Scoring Posts

- Rank all posts by their combined score.
- Recommend the highest-scoring posts to the user.

**Example: Assume two users Alice and Bob in EcoTrade**

**Users:** Alice, Bob

**Tag Vectors:**

Alice: { books: 5, reuse: 3, tech: 1 }

Bob: { books: 4, reuse: 2, tech: 0 }

**Similarity:** Alice and Bob have high cosine similarity.

**Posts:**

Post A: tags = [books, reuse]

Post B: tags = [tech]

**Alice's Recommendations:**

Post A is scored higher (matches Alice's frequent tags and is liked by Bob, a similar user).

Post B is scored lower.

**ii. Activity-Based Filtering for Marketplace Products**

Recommends products based on aggregate implicit user interactions (views, clicks, offers/requests) across all users. Products with higher engagement, weighted by action importance, are prioritized.

**Mechanics**

**1. Track Activities**

Log user actions for each product:

- **Views:** User sees a product in the marketplace.
- **Clicks:** User clicks to view product details.
- **Offers/Requests:** User initiates a trade, offer, or purchase.

**2. Calculate Relevance Scores**

Assign weights to each action (e.g., offer > click > view).

**3. Compute for Each Product**

For each product, calculate:

$$\text{Score} = (w_v \times \text{views}) + (w_c \times \text{clicks}) + (w_o \times \text{offers})$$

$w_v, w_c, w_o$  are predefined weights (e.g., 0.1, 0.3, 0.6).

Weights reflect action intent (e.g., an offer signals stronger interest than a view).

**4. Recommend Top-Scoring Products**

Rank products by their scores and recommend the highest-scoring ones to users.

**Example**

**Products:** Product X, Product Y

**Activity Counts:**

- Product X: 150 views, 50 clicks, 15 offers
- Product Y: 100 views, 80 clicks, 10 offers

**Weights:**  $w_v=0.1$ ,  $w_c=0.3$ ,  $w_o=0.6$

**Calculate Scores:**

- Product X:  $(0.1 \times 150) + (0.3 \times 50) + (0.6 \times 15) = 15 + 15 + 9 = 39$
- Product Y:  $(0.1 \times 100) + (0.3 \times 80) + (0.6 \times 10) = 10 + 24 + 6 = 40$

**Result:** Product Y (score 40) > Product X (score 39).

Recommend Product Y more prominently.

## **Chapter 4 Implementaion and Testing**

### **4.1 Tools Used**

#### **React Js**

React JS is the primary frontend library used to build the user interface of the platform. It enables the development of dynamic, component-based web applications, allowing for efficient rendering and state management. React Router is integrated for seamless navigation between pages, and Material-UI provides a set of pre-built UI components and icons for a modern look and feel. Custom CSS is used to further style and layout the application, ensuring responsiveness and accessibility.

#### **MongoDB**

MongoDB serves as the main database for the project. It is a NoSQL, document-oriented database that stores data in flexible JSON-like documents. This flexibility allows for rapid development and easy scaling. Mongoose is used as the Object Data Modeling (ODM) library, providing schema definitions, validation, and query capabilities, which help maintain data integrity and simplify interactions with the database.

#### **Node JS**

Node JS is the backend runtime environment, enabling server-side JavaScript execution. Express JS is used as the web framework to build RESTful APIs, handle routing, and manage middleware. The backend is responsible for processing requests, handling authentication (using JWT and bcrypt), managing business logic, and interacting with MongoDB to store and retrieve data.

#### **GitHub**

GitHub is a web-based version control repository and Internet hosting service. It is mostly used for code. It offers all of the distributed version control and source code management (SCM) functionality of Git as well as adding its own features. GitHub offers both plans for private and free repositories on the same account which are commonly used to host open source software projects. Projects on GitHub can be accessed and manipulated using the standard Git command-line interface and all of the standard Git commands work with it. GitHub also allows registered and non-registered users to browse public repositories on the site. Multiple desktop clients and Git plugins have also been created by GitHub and other third parties that integrate with the platform.

## 4.2 Implement Details of the modules

### User Registration, Authentication and Authorization

```
//REGISTER
router.post("/register", async (req, res) => {
  try {
    //generate new password
    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(req.body.password, salt);
    const {password,...others}=req.body
    //create new user
    // username: req.body.username,
    // email: req.body.email,
    const newUser = new User({
      ...others,
      password: hashedPassword,
    });
    //save user and respond
    const user = await newUser.save();
    res.status(200).json(user);
  } catch (err) {
    res.status(500).json(err)
  }
});

//LOGIN
router.post("/login", async (req, res) => {
  try {
    const user = await User.findOne({ email: req.body.email });
    if(!user) {

      return res.status(404).json("user not found");
    }
    const validPassword = await bcrypt.compare(req.body.password, user.password)
    if(!validPassword) {
      return res.status(400).json("wrong password")
    }
    return res.status(200).json(user)
  } catch (err) {
    res.status(500).json(err)
  }
});
```

**Listing 4.1: Code Snippet for Login and Registration**

This code handles user registration and login using Express, MongoDB, and bcrypt. During registration, the server hashes the user's password and saves the user data securely in the database. During login, it checks if the user exists and verifies the password by comparing it with the stored hash. If the credentials are correct, the user is returned; otherwise, appropriate error messages are sent. Error handling is included for both processes.

## Ads Management Module

```
router.get("/", async (req, res) => {
  try {
    const now = new Date();
    const ads = await Ad.find({
      status: "Active",
      validFrom: { $lte: now },
      validTill: { $gte: now }
    })
    .sort({ priority: -1, createdAt: -1 })
    .limit(10)
    .select("-createdAt -clickCount -impressionCount")
    .lean();
    // Track impressions for these ads
    const adIds = ads.map(ad => ad._id);
    if (adIds.length > 0) {
      await Ad.updateMany(
        { _id: { $in: adIds } },
        { $inc: { impressionCount: 1 } }
      );
    }
    res.status(200).json(ads);
  } catch (err) {
    console.error("Get public ads error:", err);
    res.status(500).json(err);
  }
});

// PUT /api/ads/:id/click - Track ad click (public)
router.put("/:id/click", async (req, res) => {
  try {
    await Ad.findByIdAndUpdate(req.params.id, { $inc: { clickCount: 1 } });
    res.status(200).json("Click tracked");
  } catch (err) {
    console.error("Track click error:", err);
    res.status(500).json(err);
  }
});
```

### Listing 4.2: Code Snippet For Ads Management

This code handles public ads display and interaction tracking. The **GET /api/ads** route retrieves up to 10 currently active ads based on date and status, sorts them by priority, removes sensitive fields, and automatically increases their impression count each time they are viewed. The **PUT /api/ads/:id/click** route increments the click count of a specific ad whenever someone clicks it, helping track engagement.

## Product Recommendation Module

```
// Calculate relevance scores and add to products
const productsWithScores = products.map(product => {
  const activities = activityData[product._id.toString()] || { view: 0,
click: 0, offer: 0 };
  const relevanceScore =
ProductActivity.calculateRelevanceScore(activities);

  return {
    ...product,
    activityCounts: activities,
    relevanceScore: relevanceScore
  };
});
// Sort by relevance score (highest first), then by creation date
productsWithScores.sort((a, b) => {
  if (a.relevanceScore === b.relevanceScore) {
    return new Date(b.createdAt) - new Date(a.createdAt);
  }
  return b.relevanceScore - a.relevanceScore;
});
```

### Listing 4.3: Code Snippet For Product Ranking

This code calculates a relevance score for each product based on its activity data (views, clicks, offers). It attaches both the activity counts and the calculated relevance score to each product. After that, the products are sorted so that those with the highest relevance score appear first. If two products have the same score, they are sorted by their creation date, with newer products shown before older ones.

## Conversation Module

```
//CREAT NEW CONVERSATION
router.post("/", async (req, res) => {
  try {
    const newConversation = new Conversation({
      members: [req.body.senderId, req.body.receiverId]
    })
    const savedConversation = await newConversation.save()
    res.status(201).json(savedConversation)
  } catch (err) {
    res.status(500).json(err)
  }
})
```

### Listing 4.4: Code Snippet for Starting Conversation

This code creates a new conversation between two users. When a POST request is sent, it takes the senderId and receiverId from the request body and stores them as members of a new conversation. The conversation is then saved to the database, and the saved

conversation details are returned with a 201 (created) status. If anything goes wrong, a 500 error response is sent.

```
router.get("/product/:productId/conversation/:userId1/:userId2", async (req, res) => {
  try {
    const { productId, userId1, userId2 } = req.params;
    const messages = await ProductMessage.find({
      productId,
      $or: [
        { senderId: userId1, receiverId: userId2 },
        { senderId: userId2, receiverId: userId1 }
      ]
    })
    .populate("senderId", "username profilePicture")
    .populate("receiverId", "username profilePicture")
    .sort({ createdAt: 1 });

    res.status(200).json(messages);
  } catch (err) {
    console.error("Error fetching conversation:", err);
    res.status(500).json(err);
  }
});
```

#### Listing 4.5 : Code Snippet to Show Conversation

This code fetches the message history between two users about a specific product. It looks for messages where both users appear as either the sender or receiver and the productId matches. The sender and receiver details are populated with their username and profile picture. The messages are sorted in ascending order by time and then returned. If any error occurs, a 500 error response is sent.

#### Friends Module

```
const newRequest = new FriendRequest({
  sender: senderId,
  receiver: receiverId
});
const savedRequest = await newRequest.save();
const populatedRequest = await FriendRequest.findById(savedRequest._id)
  .populate("sender", "username email profilePicture")
  .populate("receiver", "username email profilePicture");
```

#### Listing 4.6: Code Snippet For Adding Friends

This code creates a new friend request between two users. It first builds a FriendRequest object with the sender's and receiver's IDs, then saves it to the database. After saving, it retrieves the newly created request again and populates the sender and receiver fields with user details such as username, email, and profile picture.

```

router.put("/friend-request/:requestId/accept", async (req, res) => {
  try {
    const { userId } = req.body;
    const request = await FriendRequest.findById(req.params.requestId);

    if (!request) {
      return res.status(404).json("Friend request not found");
    }

    if (request.receiver.toString() !== userId) {
      return res.status(403).json("You can only accept requests sent to you");
    }
  }
});

```

#### Listing 4.7: Code Snippet For Accepting Friend Request

This code handles accepting a friend request. It first gets the `userId` from the request body and finds the friend request using the `requestId` from the URL. If the request doesn't exist, it returns a 404 error. Then it checks whether the logged-in user (the one sending `userId`) is actually the receiver of that friend request. If the receiver ID does not match, it blocks the action with a 403 error, ensuring only the intended user can accept the request.

```

await User.findByIdAndUpdate(request.sender, {
  $addToSet: { friends: request.receiver }
});

await User.findByIdAndUpdate(request.receiver, {
  $addToSet: { friends: request.sender }
});

```

#### Listing 4.8: Code Snippet showing Mutual Friendship

This code updates both users' friend lists when a friend request is accepted. It uses `findByIdAndUpdate` with `$addToSet` to add each user to the other's friends array making sure duplicates are not added. The first update adds the receiver to the sender's friend list, and Second update adds sender to receiver's friend list, completing the mutual friendship.

### 4.3 Testing

Software testing is a critical, continuous process used to evaluate a software product to determine if it meets specified requirements, functions correctly, and is ready for deployment. Its primary goal is to identify defects (bugs), assess the quality of the product, and ensure the final software provides the expected user experience. Basically the system is tested on two phases : Unit Testing (separately each components are tested) and System testing (components are integrated and tested).

#### 4.3.1 Unit Testing

Unit testing is a software testing method where individual components or small pieces of code such as functions, methods, or classes are tested separately to verify that each part

works correctly. It helps catch bugs early, improves code quality, and makes future changes safer. Below are the various test cases with the different scenarios with the test description, input data and the output of the test

**Table 4.1: Test Case for Login.**

| Test Case | Test Description        | Input                                                                               | Expected Output                                | Status |
|-----------|-------------------------|-------------------------------------------------------------------------------------|------------------------------------------------|--------|
| 1         | Valid user registration | Valid email, password, personal details                                             | User created successfully                      | PASS   |
| 2         | Email format validation | Invalid email format                                                                | Error: Invalid email format                    | PASS   |
| 3         | Admin Login             | Email: <a href="mailto:nirmal@nirmal.com">nirmal@nirmal.com</a><br>Password: nirmal | User Logged with Full admin Access and Cotrols | PASS   |
| 4         | Login                   | User: <a href="mailto:aaa@aaa.com">aaa@aaa.com</a><br>Password: aaaaaa              | Successful User Login                          | PASS   |

**Table 4.2: Test Case for Post Feed.**

| Test Case | Test Description  | Input                                                                   | Expected Output             | Status |
|-----------|-------------------|-------------------------------------------------------------------------|-----------------------------|--------|
| 1         | Post Creation     | Fill Out the fill and click on Share button                             | Post Created Successful     | PASS   |
| 2         | Post Liking       | Clicking on the reaction Button                                         | Reaction Count Increase     | PASS   |
| 3         | Post Commenting   | Click in show comments , typing the comment and clicking in post button | Success Post Creation       | PASS   |
| 4         | Editing the posts | Clicking on my post and edit button and editing details                 | Post Editing with new Entry | PASS   |

**Table 4.3: Other Test Cases**

| Test Case | Test Description           | Input                                            | Expected Output                                    | Status |
|-----------|----------------------------|--------------------------------------------------|----------------------------------------------------|--------|
| 1         | Profile Editing            | Clicking on Edit Profile and filling new details | Profile Should be edited                           | PASS   |
| 2         | Viewing Trade Centre       | Clicking on Trade centre                         | Display the recent completed and incomplete trades | PASS   |
| 3         | Viewing Friends            | Clicking on Friends icon                         | Show users friends                                 | PASS   |
| 4         | Accepting Friends Requests | Friends → Pending → Accept                       | Friends Should be added                            | PASS   |
| 5         | Logout                     | Click on Logout                                  | Login Page should be Displayed                     | PASS   |

**4.3.2 System Testing**

System testing is a critical phase in the software development lifecycle that ensures the developed EcoTrade system functions as intended and meets the specified requirements. It involves testing the entire integrated system as a whole, rather than individual components, to verify its correctness, reliability, and performance. The main goal of system testing is to identify any defects, inconsistencies, or unexpected behaviors before the system is deployed for actual use.

**Table 4.4: System Test Cases For Friend and Messaging**

| Test Case | Test Description         | Input                                                              | Expected Output                                       | Status |
|-----------|--------------------------|--------------------------------------------------------------------|-------------------------------------------------------|--------|
| 1         | Sending messages         | Search User → Add Friend<br>My Friends → Send Message              | Friend Request must be sent<br>Message should be sent | PASS   |
| 2         | Item claiming            | Marketplace → View Order<br>→ Enter Offer → Send Message           | Order Message is sent                                 | PASS   |
| 3         | Advertisement Management | Admin Login → Admin<br>→ Advertisement → Add new<br>→ Fill details | Ad should be displayed in Home page                   | PASS   |

## 4.4 Result Analysis

### Post Creation and Recommendation

The Post Creation and Recommendation module is a central feature designed to foster user engagement and community interaction within the platform. It enables users to express themselves, share updates, and participate in discussions, while also leveraging intelligent algorithms to personalize content recommendations.

Users can create posts through an intuitive interface that supports multiple content types. The post editor allows users to:

- **Write text updates:** Share thoughts, experiences, or announcements.
- **Upload images and media:** Attach photos, videos, or documents to enrich posts.
- **Tag friends or products:** Mention other users or reference marketplace items to increase visibility and relevance.
- **Set privacy options:** Choose whether posts are public, visible to friends, or restricted to specific groups.

Once a post is created, it is stored in the database with metadata such as the author, timestamp, attached media, and any tags or mentions. The system ensures that posts are displayed in real-time on the user's feed and are accessible to others based on the selected privacy settings.

The module supports a variety of interaction mechanisms:

**Likes and reactions:** Users can express appreciation or feedback on posts.

**Comments:** Facilitate discussions and conversations around shared content.

### Marketplace and Product Listing

The Marketplace Module is a comprehensive feature designed to facilitate eco-friendly trading among users. It serves as a digital marketplace where individuals can list products for sale, exchange, or giveaway, fostering a community-driven approach to buying and selling. Users can easily create product listings by providing essential details such as product name, category, condition (brand new, used, etc.), description, images, and desired transaction type (sale, exchange, or giveaway). The listing form is user-friendly and supports multiple images, allowing sellers to showcase their items effectively. Each product is stored in the database with metadata including the seller's information, posting date, and transaction preferences.

### **Browsing and Search**

The marketplace offers robust browsing and search capabilities. Users can filter products by category, price range, location, and transaction type. Advanced search features enable users to find specific items quickly, while sorting options (e.g., newest, lowest price, most popular) enhance the browsing experience. Product cards display key information at a glance, and clicking on a product reveals a detailed view with full descriptions, images, seller contact details, and transaction history.

### **Buying and Claiming**

For products listed for sale, users can initiate the buying process by making offers. The system supports negotiation, allowing buyers to propose a price and sellers to accept, reject, or counter-offer. For giveaways and exchanges, users can submit claims or exchange requests, specifying their interest and, if applicable, the item they wish to offer in return. The module tracks all offers and claims, ensuring that both parties are informed of the status and can communicate directly through integrated messaging.

### **Offers and Claims Management**

All offers and claims are managed within the module, providing transparency and accountability. Sellers can view incoming offers and claims, respond to them, and update the product status accordingly. Buyers and claimants receive notifications about the progress of their transactions, including acceptance, rejection, or requests for further negotiation. The system maintains a history of all interactions, which helps resolve disputes and builds trust among users.

### **Searching Friends and Adding Friends**

The Searching Friends and Adding Friends module is a cornerstone of the platform's social networking capabilities. It enables users to discover and connect with others, fostering a sense of community and collaboration. Users can search for other members using various criteria such as name, interests, or location. The search functionality is designed to be fast and intuitive, often leveraging autocomplete and filtering to help users find relevant profiles quickly.

Once a user identifies someone they wish to connect with, they can send a friend request. The recipient receives a notification and can choose to accept or reject the invitation. Accepted requests add both users to each other's friends list, unlocking additional features such as private messaging, viewing friends-only posts, and participating in exclusive groups or events. The module also allows users to manage their connections, including removing friends or blocking users if necessary.

Behind the scenes, the system maintains a database of friend relationships, ensuring that connections are mutual and up-to-date. The module may also suggest new friends based on shared interests, mutual connections, or recent interactions, using recommendation algorithms to enhance user engagement. By enabling users to build and manage their social network, this module strengthens the platform's community and encourages ongoing participation.

### **Ad Management by Admin**

The Ad Management by Admin module equips platform administrators with robust tools for overseeing advertisements. Ads are a vital part of the platform's ecosystem, serving both as a revenue stream and a means to promote relevant products, services, or events to users. Admins can create new ads, specifying details such as title, description, target audience, display duration, and associated media.

Once created, ads can be edited to update content or adjust targeting parameters. Admins have the authority to approve ads before they go live, ensuring that all promotional material adheres to platform guidelines and is appropriate for the user base. If an ad is found to be irrelevant, inappropriate, or outdated, admins can remove it from circulation with a single action. The module includes analytics features, allowing admins to track ad performance, such as views, clicks, and conversions. This data helps optimize future ad placements and ensures that users are exposed to high-quality, relevant content. By maintaining control over advertisements, admins help preserve the integrity of the platform, enhance user experience, and support the platform's financial sustainability.

### **Admin Module**

The Admin Module is the backbone of platform governance and security. It provides administrators with comprehensive controls to manage users, monitor activities, and enforce rules. Admins can view and edit user profiles, activate or deactivate accounts, and issue warnings or bans to users who violate community guidelines. This ensures that the platform remains a safe and welcoming environment for all participants.

In addition to user management, admins oversee marketplace activities, including product listings, offers, claims, and transactions. They can intervene in disputes, resolve issues, and ensure that all trading activities comply with platform policies. The module may also provide tools for content moderation, allowing admins to review posts, comments, and messages for inappropriate material.

## **Chapter 5: Conclusion**

### **5.1 Conclusion**

EcoTrade addresses the rising demand for sustainable and community-focused product sharing by providing a modern, responsive, and location-aware web platform. It enables users to buy, exchange, or give away products within their local communities, making the process more convenient and environmentally friendly. Built using the MERN stack, the system ensures high performance, scalability, and smooth user interactions.

By following the Agile development methodology, EcoTrade evolves continuously through iterative updates and user feedback, allowing new features and improvements to be integrated effectively. Its automated content moderation system helps maintain a clean and secure marketplace, while the recommendation engine offers personalized suggestions that enhance the overall user experience. Additionally, the platform's secure architectural design ensures user data protection and safe transactions.

Overall, EcoTrade not only simplifies community-based trading but also actively encourages sustainable practices by promoting product reuse, reducing waste, and fostering a culture of environmental responsibility among its users. The system also allows creating post , interacting the posts , chatting , adding friends and social Engagements

### **5.2 Outcomes**

With the implementation of EcoTrade, the platform successfully creates a more efficient, user-friendly, and eco-friendly system for exchanging products within local communities. It makes it easier for users to find the items they need, promotes reuse over buying new products, and strengthens community collaboration through digital communication tools. The major outcomes of the system include the following:

- EcoTrade organizes all items into well-defined categories, making it simple for users to search, filter, and browse available products. This structured layout helps users quickly find the items they want without confusion, improving overall accessibility and user satisfaction.
- The platform includes a direct messaging feature that allows users to communicate instantly with product owners. This helps clarify details, negotiate exchanges, and build trust within the community, creating a more interactive and cooperative environment.

- EcoTrade uses collaborative filtering to analyze user behavior and preferences. Based on these insights, the system provides smart recommendations tailored to each user. This increases the chances of finding relevant products and enhances the browsing experience.
- The platform displays rich, interactive pages for each product and user profile, including images, descriptions, availability, and user activity. These detailed views help users make informed decisions and engage more deeply with the platform.

### **5.3 Future Recommendations**

EcoTrade can be further improved by adding smarter features, increasing user engagement, and expanding its community-focused capabilities. These enhancements will strengthen the platform's usability, sustainability impact, and long-term scalability. Following features can be added and implemented to enhance EcoTrade:

- Develop a mobile app for better accessibility and engagement.
- Making the system more interactive with features of video and reels uploading,
- Integrate map-based product discovery,
- -provide an admin analytics dashboard for effective monitoring.

## References

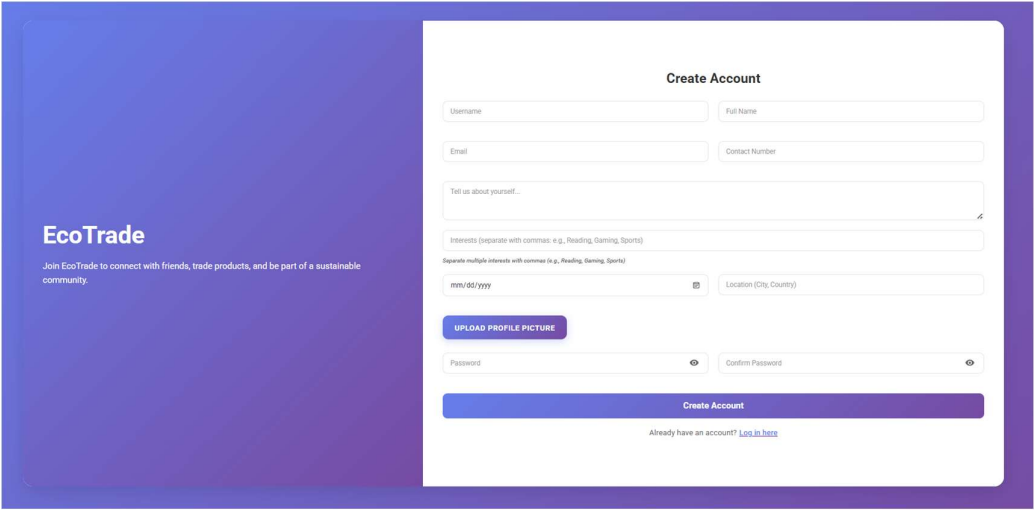
- [1] A. Sundararajan, *The Sharing Economy: The End of Employment and the Rise of Crowd-Based Capitalism*, Cambridge: MIT Press, 2016.
- [2] J. hamari, S. Mimmi and A. Ukkonen, "The Sharing Economy: Why People Participate in Collaborative Consumption," *Journal of the Association for Information Science and Technology*, vol. 67, no. 09, pp. 2047-2059, 2016.
- [3] M. Zeng and J. Wei, "Sustainable consumption and sharing economy in the digital era," *Journal of Cleaner Production*, vol. 270, no. 12, pp. 122-217, Jan 2020.
- [4] M. Inc, "MongoDB Inc," [Online]. Available: <https://www.mongodb.com/docs/>. [Accessed 5 June 2025].
- [5] R. Botsman and R. Rogers, *What's Mine Is Yours: The Rise of Collaborative Consumption*, New York: HarperBusiness, 2010.
- [6] E. Miller and L. Buys, "'A sustainability assessment: Toward conservation framework," *Sustainability Science*, vol. 3, no. 1, pp. 55-66, 2018.
- [7] B. Cohen and J. Kietzmann, "Ride On! Mobility Business Models for the Sharing Economy," *Organization & Environment*, vol. 27, no. 3, pp. 279-296, 2014.
- [8] G. Raju, "Impact of Social Media," *Simplirun*, pp. 1-8, 2025.
- [9] W. Mangold and D. Faulds, "Social media: The new hybrid element of the promotion mix," *Business Horizons*, vol. 52, no. 4, pp. 357-365, 2009.
- [10] J. Kietzmann, K. Hermkens, I. McCarthy and B. Silvestre, "Social media? Get serious! Understanding the functional building blocks of social media," *Business Horizons*, vol. 54, pp. 241-251, 2011.
- [11] A. M. Kaplan and M. Haenlein, "Users of the world, unite! The challenges and opportunities of Social Media," *Business Horizons*, vol. 53, no. 1, pp. 59-68, 2010.

- [12] D. M. Boyd and N. B. Ellison, "Social network sites: Definition, history, and scholarship," *Journal of Computer-Mediated Communication*, vol. 13, no. 1, pp. 210-230, 2007.
- [13] Y. Benkler, "Sharing Nicely: On Shareable Goods and the Emergence of Sharing as a Modality of Economic Production," *Yale Law Journal*, vol. 114, no. 2, pp. 273-358, 2004.
- [14] M. S. TUTORIAL, "MERN Stack Tutorial," 2023. [Online]. Available: <https://www.mern.io/>. [Accessed 07 June 2025].
- [15] A. Kaplan and M. Haenlein, "Users of the world, unite! The challenges and opportunities of Social Media," *Business Horizons*, vol. 53, no. 1, pp. 59-68, 2010.

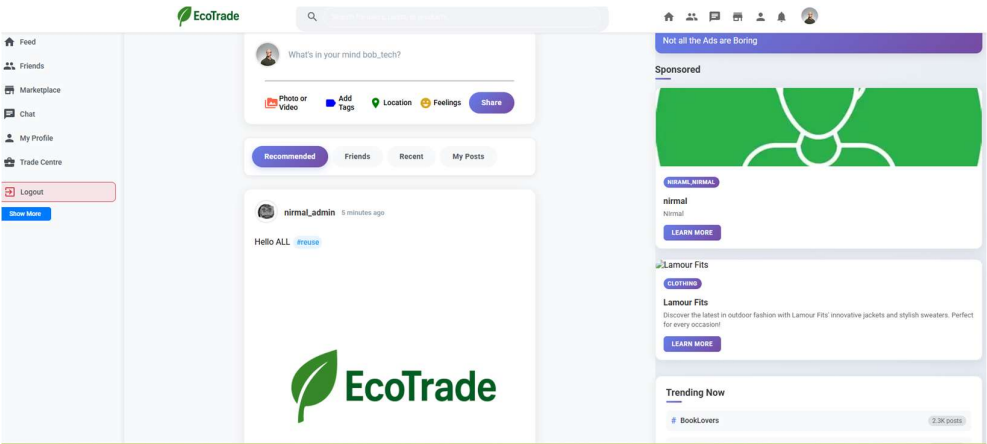
# Appendices

## Screenshots

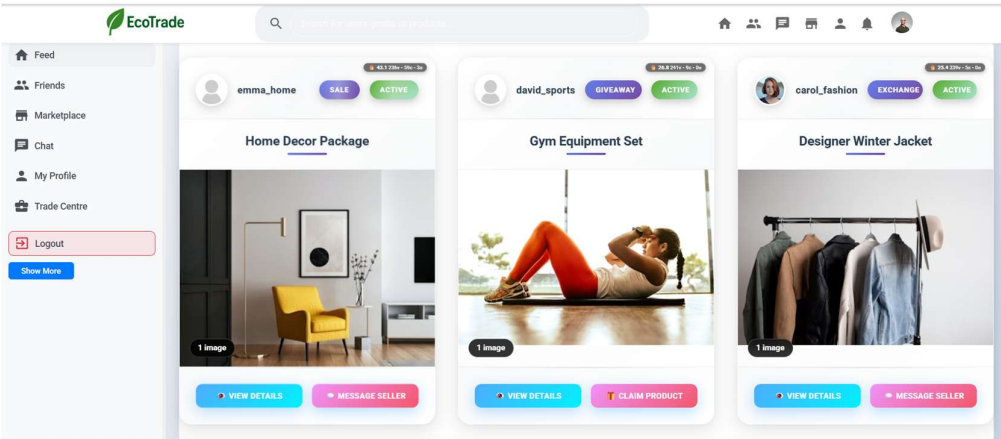
### Login Page



### Home Page



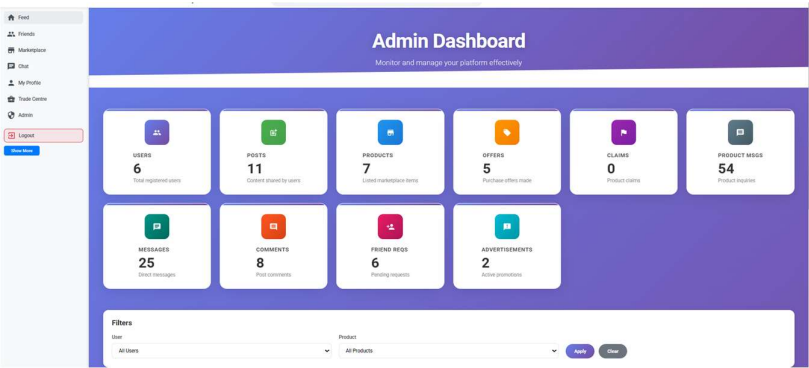
### Marketplace



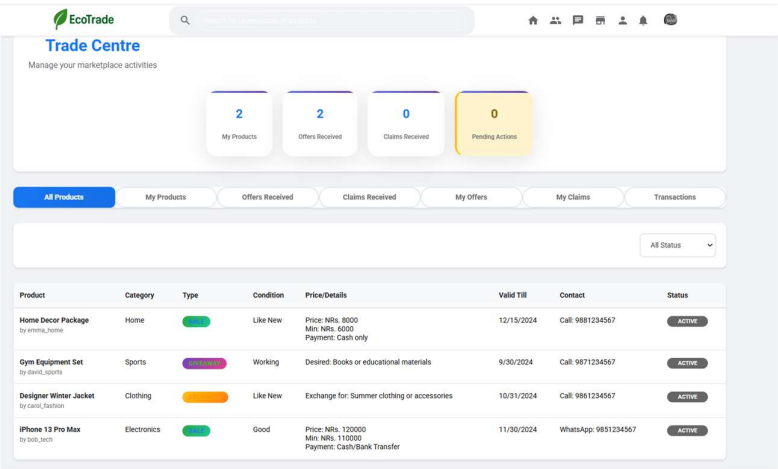
Profile Page



Admin Dashboard



Trade Centre



## Source Code

```
useEffect(() => {
  fetchFriends();
  fetchPendingRequests();
  fetchSentRequests();
}, []);

const fetchFriends = async () => {
  try {
    setLoading(true);
    const res = await apiClient.get(`/friends/friends/${user._id}`);
    setFriends(res.data);
  } catch (err) {
    console.error("Error fetching friends:", err);
  } finally {
    setLoading(false);
  }
};

const fetchPendingRequests = async () => {
  try {
    const res = await apiClient.get(`/friends/friend-requests/received/${user._id}`);
    setPendingRequests(res.data);
  } catch (err) {
    console.error("Error fetching pending requests:", err);
  }
};

const fetchSentRequests = async () => {
  try {
    const res = await apiClient.get(`/friends/friend-requests/sent/${user._id}`);
    setSentRequests(res.data);
  } catch (err) {
    console.error("Error fetching sent requests:", err);
  }
};

const searchUsers = async (query) => {
  if (query.trim().length < 2) {
    setSearchResults([]);
    return;
  }

  try {
    const res = await apiClient.get(`/friends/search?q=${query}`);
    // Filter out current user and existing friends
    const filteredResults = res.data.filter(
      (searchUser) =>
        searchUser._id !== user._id &&
        !friends.some(friend => friend._id === searchUser._id) &&
        !pendingRequests.some(req => req.sender._id === searchUser._id) &&
        !sentRequests.some(req => req.receiver._id === searchUser._id)
    );
    setSearchResults(filteredResults);
  } catch (err) {
    console.error("Error searching users:", err);
  }
};

const sendFriendRequest = async (receiverId) => {
  try {
    const response = await apiClient.post("/friends/friend-request", {
      senderId: user._id,
      receiverId: receiverId
    });

    console.log("Friend request sent successfully:", response.data);

    // Remove from search results and add to sent requests
    setSearchResults(prev => prev.filter(u => u._id !== receiverId));
    fetchSentRequests();

    alert("Friend request sent successfully! The user will receive a notification.");
  }
};
```

```

    } catch (err) {
      console.error("Error sending friend request:", err);
      const errorMessage = err.response?.data || "Error sending friend request. Please try again.";
      alert(errorMessage);
    }
  };

```

```

const acceptFriendRequest = async (requestId) => {
  try {
    await apiClient.put(`/friends/friend-request/${requestId}/accept`, {
      userId: user._id
    });
  }

```

```

  /* Friends Tab */
  {activeTab === "friends" && (
    <div className="friendsTab">
      {friends.length === 0 ? (
        <div className="noFriends">
          <h3>No Friends Yet</h3>
          <p>Start by searching for users and sending friend requests!</p>
          <button
            className="searchUsersBtn"
            onClick={() => setShowSearch(true)}
          >
            Search Users
          </button>
        </div>
      ) : (
        <div className="friendsGrid">
          {friends.map(friend => (
            <div key={friend._id} className="friendCard">
              <img
                src={friend.profilePicture || "/assets/person/noAvatar.png"}
                alt=""
                className="friendImg"
              />
              <div className="friendInfo">
                <h4>{friend.username}</h4>
                <p>{friend.email}</p>
              </div>
              <div className="friendActions">
                <button
                  className="messageBtn"
                  onClick={() => handleMessage(friend._id)}
                >
                   Message
                </button>
                <button
                  className="viewProfileBtn"
                  onClick={() => handleViewProfile(friend.username)}
                >
                   Profile
                </button>
                <button
                  className="removeFriendBtn"
                  onClick={() => removeFriend(friend._id)}
                >
                   Remove
                </button>
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  )}

  /* Pending Requests Tab */
  {activeTab === "pending" && (
    <div className="pendingTab">
      {pendingRequests.length === 0 ? (
        <div className="noRequests">
          <h3>No Pending Friend Requests</h3>
          <p>When someone sends you a friend request, it will appear here.</p>

```

```
</div>
): (
<div className="requestsGrid">
{pendingRequests.map(request => (
  <div key={request._id} className="requestCard">
    <img
      src={request.sender.profilePicture || "/assets/person/noAvatar.png"}
      alt=""
      className="requestImg"
    />
    <div className="requestInfo">
      <h4>{request.sender.username}</h4>
      <p>{request.sender.email}</p>
      <p className="requestDate">
        Sent: {new Date(request.createdAt).toLocaleDateString()}
      </p>
    </div>
    <div className="requestActions">
      <button
        className="acceptBtn"
        onClick={() => acceptFriendRequest(request._id)}
      >
        ✓ Accept
      </button>
      <button
        className="rejectBtn"
        onClick={() => rejectFriendRequest(request._id)}
      >
        ✗ Reject
      </button>
      <button
        className="viewProfileBtn"
        onClick={() => handleViewProfile(request.sender.username)}
      >
        👤 Profile
      </button>
    </div>
  )})}
</div>
))}
</div>
) : (
/* Sent Requests Tab */
{activeTab === "sent" && (
  <div className="sentTab">
    {sentRequests.length === 0 ? (
      <div className="noSentRequests">
        <h3>No Sent Friend Requests</h3>
        <p>You haven't sent any friend requests yet.</p>
      </div>
    ) : (
      <div className="sentRequestsGrid">
        {sentRequests.map(request => (
          <div key={request._id} className="sentRequestCard">
            <img
              src={request.receiver.profilePicture || "/assets/person/noAvatar.png"}
              alt=""
              className="sentRequestImg"
            />
            <div className="sentRequestInfo">
              <h4>{request.receiver.username}</h4>
              <p>{request.receiver.email}</p>
              <p className="sentRequestDate">
                Sent: {new Date(request.createdAt).toLocaleDateString()}
              </p>
            </div>
            <div className="sentRequestActions">
              <button
                className="cancelBtn"
                onClick={() => cancelFriendRequest(request._id)}
              >
                ✗ Cancel
              </button>
```

```

        <button
          className="viewProfileBtn"
          onClick={() => handleViewProfile(request.receiver.username)}
        >
           Profile
        </button>
      </div>
    </div>
  </div>
)
}
}
</div>
</div>
</Layout>
);
}

```

*// Ensure every axios call points at the API server (dev proxy or prod URL)*

```

const API_BASE_URL =
  process.env.REACT_APP_API_URL ||
  process.env.REACT_APP_API_BASE_URL ||
  "/api";

```

```

axios.defaults.baseURL = API_BASE_URL;

```

*// Prevent runtime errors if any bundle references process in the browser*

*// without CRA inlining it*

```

if (typeof window !== "undefined" && typeof window.process === "undefined") {
  window.process = { env: {} };
}

```

```

ReactDOM.render(
  <React.StrictMode>
    <AuthProvider>
      <NotificationProvider>
        <App />
      </NotificationProvider>
    </AuthProvider>
  </React.StrictMode>,
  document.getElementById("root")
);

```

*// 2. Collaborative score: weight based on similar users who actually liked this post*

```

let collabScore = 0;
const likeSet = new Set(Array.isArray(postLikes) ? postLikes.map(String) : []);

```

```

similarUsers.forEach((u) => {
  const similarUserId = String(u.userId);
  const similarity = u.similarity || 0;

  if (similarUserId !== userId && similarity > 0) {
    // Check if similar user actually liked this post
    if (likeSet.has(similarUserId)) {
      collabScore += similarity; // Full weight if they liked it
    }
  }
}

```

*// Additional weight if similar user is the author (content from similar users)*

```

if (String(postUserId) === similarUserId) {
  collabScore += similarity * 0.5; // Half weight for authorship
}
}
});

```

*// Combine scores: 70% content-based, 30% collaborative*

```

const finalScore = contentScore * 0.7 + collabScore * 0.3;

```

```

return {
  contentScore: Number(contentScore.toFixed(4)),
  collabScore: Number(collabScore.toFixed(4)),
  finalScore: Number(finalScore.toFixed(4)),
  tags: postTags,
}

```

```
};  
};  
  
// Full recommendation function with detailed scoring  
const recommendPostsForUser = async (userId, limit = 10) => {  
  // Get user's tag interaction record  
  const userRecord = await UserTagInteraction.findOne({ userId });  
  if (!userRecord) {  
    // Return empty array if user has no interaction history  
    return [];  
  }  
}
```